Action-Effect: An Empirical Protocol for Interchanging Effect Descriptions Without Conflating Evidence and Permission

Ond Holdings Inc.
Toronto, Ontario, Canada

Working draft
First drafted 18 October 2026; revised 19 October 2026
America/Toronto, EDT, UTC-04:00

Working paper for public review

This standalone edition accompanies `manuscript.md`. Retrieval and run times use UTC. America/Toronto is the editorial timezone, not execution geolocation. The Spec release and this paper have separate publication dates.

Abstract

Action-Effect is a versioned interchange for coupled effect descriptions bound to captured arguments, implementation declarations, context, evidence, and explicit coverage. We implement a finite GitHub issue-comment profile with a JavaScript producer, a separately written Python checker, current OPA authorization, and an occurrence-reserving dispatch gate. Its technical contract specifies canonical preimages, ordered refusals, and read-only recovery after uncertain delivery. Factorized measurement separates mapping, reference-input authorization, admission, observation, task utility, and cost. The main comparator is a complete structured adapter sharing policy, projection semantics, and dispatch controls. Both strategies matched 108 native lane instances from 55 authored cases in 22 intent groups and all 52 eligible decisions. A controlled live run matched 32 planned checks and created 12 comments, including four recovered after deliberately discarded acknowledgments. Accepted-input offline median cost was 3.97% higher for the protocol. A separate read-only study used an existing Azure model and the official GitHub MCP server on six maintainer goals. In its first run, all six completed loop and schema checks. Only one extracted Markdown body fit 6000 bytes, and none of the complete JSON responses met the original delivery contract; review identified conflicting harness instructions. Internal fact review marked 27 of 30 facts met, two partial and one incorrect. An amended repeat completed two tasks and retained four byte-budget failures; all six fully packaged repeat outputs nevertheless fit the larger comment-profile limit. Its separate review marked 28 facts met, two partial and none incorrect, without reclassifying failed loops. A subsequent owned-resource study delivered the two eligible reports through SDK and official MCP, then exercised six separate process/acknowledgment faults. Twenty fresh workers made five comment calls; all eight duplicate probes made none. Three delivered effects were recovered read-only, while one pre-send interruption correctly remained unknown. Human utility remained unmeasured. These observations show why semantic mapping, useful output, and delivered effects need distinct evidence. The contribution is an implemented interchange, factorized evaluation, and inspectable conditional recovery contract, with reproducible parity, cost, and integration findings. The evidence is same-project and bounded; provider descendants and external validation remain outside the observations.

1 Problem and Contribution

A tool name does not identify every consequence, and a worrying string does not establish a harmful occurrence. An interchange must preserve coupled operation, resource, destination, data, quantity, applicability, and uncertainty without confusing a declaration with authority. Existing interfaces supply complementary facilities: MCP tool-level hints [S1], AuthZEN decision exchange [S2], OAuth RAR permission objects [S3], and structured OPA/Cedar policy evaluation [S4, S5, S6, S7]. Action-Effect addresses description transfer and checking between native capture and authorization, preserving these systems' responsibility for authority.

We present three implemented contributions:

1. **Versioned coupled interchange and finite profile.** Exact native/context preimages, implementation declarations, evidence, quantities, coverage and unresolved descendants cross a JavaScript/Python boundary. Equivalent SDK and MCP captures retain distinct invocation identities while producing the same supported direct-effect projection.

2. **Factorized evaluation with a complete baseline.** Literal mapping references, pure authorization controls, integrated outcomes and cost distinguish correct semantics from a decision that masks a mapping defect. Retained reports, source pins and replay make comparisons inspectable, including failed attempts.

3. **Conditional dispatch and recovery contract.** Durable occurrence reservation, current-policy checks, exact-ID readback and no-resend recovery have explicit transitions, failure cuts, trust dependencies and counterexamples. Task correctness is evaluated separately from request binding and delivery.

The measured result is cross-language mapping parity with a small accepted-input cost increase, controlled recovery and concrete native-integration findings. The first real-model run also exposed an output-contract mismatch that loop and citation checks did not detect. The amended repeat retained four task-budget failures even though all six final comment preparations fit the native profile. Actual delivery then preserved the two-eligible/four-withheld split; real-process recovery retained uncertainty instead of resending. These findings motivate the engineering question: when does interoperable description checking justify its additional integration and runtime cost relative to a complete direct adapter?

1.1 Running Operation: A Maintainer's Comment

A maintainer prepares a release-verification note for a dedicated issue. The illustrative body `abc` exposes exact byte bindings. A useful report also needs source-based checks of its technical conclusions, as the real maintainer tasks in Section 8 demonstrate.

Let N be an SDK capture with lane `sdk`, owner `OndCo`, repository `Action-Effect-Bench`, issue number 1 and that body. Context C supplies a principal, `synthetic-public` and `github-comment-bridge/1`. An MCP capture nests the same parameters under `call.arguments` and names `add_issue_comment`. The direct requested mutation is the same, but the native JSON captures are different and should have different digests.

The checked projection binds the exact issue, constructed API endpoint, three UTF-8 bytes and their SHA-256. It grants nothing. The direct comment is known, while recipients, delivery and provider descendants are not observed. Suppose current policy grants this repository/issue/class

Change	Obligation	Interpretation
Body `abc` to `abd`	Native/body/description binding	Recheck exact bytes; old evidence is inapplicable
SDK to MCP capture	Invocation identity	Different digest can retain equal direct projection
Different issue	Resource, endpoint, grant	Authorize the new coupled tuple
Policy revoked	Current authority	Accepted description cannot override denial
Malformed policy	Processing	Withhold as failure, not semantic denial
ACK absent after possible POST	Occurrence and observation	Unknown; no automatic resend
Identical text from another actor	Observation binding	Body equality alone is insufficient
False technical conclusion	Original task utility	Correct mapping does not make the task correct

Table 1: One running operation, different obligations. These explanatory counterfactuals are not additional experimental cases.

triple, permits at least three bytes and explicitly accepts partial direct scope. The proposal can receive allow. Revoking that policy gives deny. Retaining the grant but not accepting partial scope gives review. Adding an executable argument makes native input invalid; an unreadable policy is a processing failure. These outcomes require different operator responses.

The gate reserves a caller-selected occurrence. If a successful POST's response is lost, the requested task has not changed, but delivery knowledge has. Assigning another occurrence and resending could duplicate the effect. The original reservation is retained and reconciliation only reads. A matching current object is stronger evidence than local admission, but not a complete provider execution history.

1.2 Why Keep the Tuple Intact?

For grants $(A, 1, \text{public})$ and $(B, 2, \text{internal})$, independent membership in allowed repository, issue and class sets would admit $(A, 2, \text{internal})$. Neither grant asserts it. One complete row must match. The Rego policy preserves that relationship, as any competent structured baseline can. The interchange obligation is not to destroy an existing policy capability, not to invent tuple-based authorization.

2 Semantic and Trust Model

An effect is a coupled tuple of category, operation, resource, destination, data class, quantities, and modality. A description adds the effective profile, mapping revision, invocation/implementation/context bindings, evidence, coverage, and explicit unresolved facts. It does not redefine the current grant or the execution occurrence. Known, none, and unknown remain distinct. Fields from different alternatives cannot be recombined into an unasserted permission.

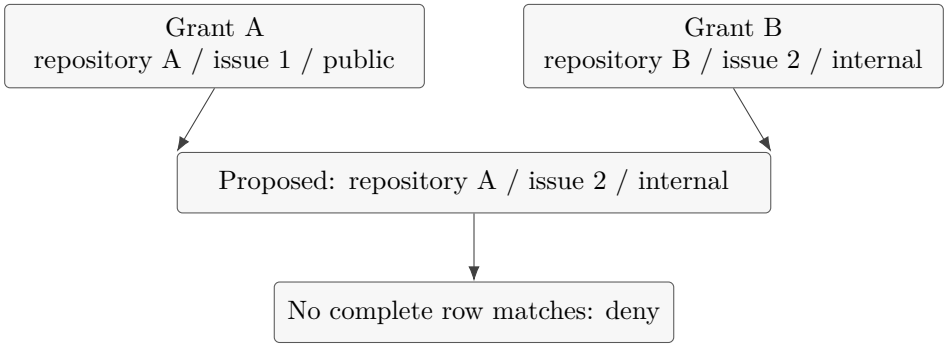


Figure 1: Coupling is a row-level obligation. Taking independent field sets creates combinations that neither grant asserted.

2.1 Evidence Is Not Permission

Description processing establishes supported interpretation; a checker can establish only the particular claim covered by its evidence and assumptions. Authorization uses separately authenticated facts and current policy. A final gate controls dispatch, while a provider observation covers an actual occurrence. None of these states implies all the others. Description acceptance remains `authority: not_evaluated`.

Missing required evidence withholds the affected execution but is not a correctly recognized semantic block. An applicable trusted denial survives unrelated uncertainty. Human review can authorize a deliberately limited scope; it cannot make an invalid proof valid. Acceptance of a partial direct-effect profile for one owned test resource cannot silently downgrade another policy's requirement for complete descendant coverage.

2.2 Conditional Guarantees and Assumptions

If a sound checker establishes $E \subseteq U$, authority permits U, and execution stays mediated under the same assumptions, covered admitted effects respect that authority. For a policy expressible as an allowed set B, the implication from $E \subseteq U$ and $U \subseteq B$ to $E \subseteq B$ is elementary. It identifies premises an implementation must discharge; a registry signature does not discharge them. History, aggregate budgets, and temporal constraints are not generally independent per-tuple membership tests. An observed trace also does not establish a complete upper bound on possible behavior.

2.3 Threat and Observation Boundaries

Relevant faults include body/context/implementation drift, stale evidence, untrusted declarations, tuple recombination, changed authority, duplicate occurrences, and uncertain delivery. Arbitrary hidden SaaS behavior, covert channels, and unmediated effects outside the declared universe are not covered by the finite pilot.

2.4 Identities and Canonical Preimages

Define $\mathcal{J}(x)$ as the profile's canonical JSON UTF-8 bytes and $\mathsf{H}(x) = \mathrm{SHA256}(\mathcal{J}(x))$. For an already specified byte sequence, $\mathsf{H}_b(b) = \mathrm{SHA256}(b)$. Hashing a JSON string containing `abc` is not hashing its three content bytes. The restricted core domain rejects duplicate decoded names and

invalid Unicode scalars, and supports safe integer JSON tokens without fraction/exponent syntax. Arrays retain order, object keys are canonically sorted, and strings are not normalized. Decision quantities use exact decimal strings, not floating-point conversions.

Semantic action, native capture, description record, execution occurrence and provider object are different identities. The core leaves the application's semantic action and occurrence naming intact. Table 2 lists the implemented bindings. Here P is the full installed profile manifest.

Table 2: Exact identity targets. Equal bytes or values establish the indicated identity, not authority or factual truth.

Identity	Preimage/source	Meaning
Profile	$\mathsf{H}(P)$	Same complete manifest
Invocation	$\mathsf{H}(N)$	Same lane, wrapper and arguments
Context	$\mathsf{H}(C)$	Complete supplied context, including extra fields
Implementation	Hash of ID plus declaration kind	Same declaration, not SaaS attestation
Mapping	Profile URI, fragment mapping, revision 1, $\mathsf{H}(P)$	Selected mapping contract
Declaration artifact	$\mathsf{H}(\{N, C, \mathsf{H}(P)\})$ with named fields	Same joint declaration inputs
Record suffix	$\mathsf{H}(\{\text{native} : N, \text{context} : C\})$	Same pair; profile revision is not in this suffix
Description	$\mathsf{H}(D)$	Entire description, evidence and unknowns
Body	$\mathsf{H}_b(\text{UTF8}(\textit{body}))$	Exact bytes, without trimming/normalizing
Ledger binding	Hash of native/context pair	Same pair for the reserved occurrence; not policy
Ledger filename	Byte hash of occurrence string plus `.json`	Local storage address
Comment ID	Provider response/readback integer	Provider object, not caller occurrence

Local invocation/context/implementation/declaration URNs are configured selectors, not fetch instructions. The locator string, selected object and enclosing file have different possible hashes. Only the declared preimage is meaningful. The mapping fragment likewise selects local contract semantics, not ambient URL-fragment resolution.

Record ID alone cannot identify acceptance across profile changes: its suffix binds N, C, while the complete description binds P. The ledger binding also omits policy deliberately. A request can remain identical while authority narrows. These distinctions prevent reuse of one digest as a universal semantic, execution or permission identity.

2.5 Trust and Refusal Ownership

Untrusted inputs, malformed descriptions, false declarations, changed arguments, stale prepared data and misleading current matches are in scope. A compromised host cannot make its own account trustworthy by hashing it. Gate code, credentials, clocks, filesystem, policy configuration and provider adapter are trusted components, with the responsibilities in Table 3.

Table 3: Trust ledger and residual failures. A role name does not establish independence.

Fact	Acquisition	Trusted owner	Residual risk
Native invocation	Allowlisted snapshot	Capture/gate	Bypassed or dishonest capture
Semantics	Installed profile and checker	Installer/maintainer	Incorrect shared mapping
Live principal	Authenticated provider lookup	Credential owner/provider	Missing or different identity
Data class	Configured context	Classification process	No content classifier
Current grant	Policy load after preflight	Policy owner/OPA wrapper	Malformed or stale upstream facts
Occurrence	Exclusive creation/transitions	Local filesystem	Other hosts, deletion, stale locks
Current content	ID/issue/body/actor GET	Bound provider adapter	Ambiguous/missing evidence
Run identity	Sources, reports and journal hashes	Operator/trusted pins	Consistent fabrication
Task success	Original criteria and source review	Task evaluator	Correctly mapped false statement

A policy owner can resolve a grant, but cannot repair malformed input for a parser or manufacture a receipt. One scalar risk score would lose that refusal ownership. Multiple roles can belong to one organization without becoming independent checks.

2.6 Conditional Invariants and Counterexamples

These code-level invariants connect checker and gate behavior to explicit assumptions. Each argument states its dependency and a counterexample when that dependency is removed. The observation universe is the direct comment; unresolved descendants remain unresolved.

I1: accepted-description consistency. Fix trusted P, unchanged N, C, deterministic projection/description functions and correct comparison. Python `check` acceptance implies

$$\mathcal{J}(D) = \mathcal{J}(\mathrm{Describe}_P(N, C, \mathrm{Project}_P(N, C))).$$

The consumer reconstructs every expected field, including coverage, evidence and unknowns, then compares the complete canonical values. A changed body hash or extra array entry cannot pass merely by being well-shaped. The conclusion is consistency with a finite installed mapping. If that mapping omits a real webhook, two correct implementations can agree on an incomplete description.

I2: decision congruence under matched preparation. Identical complete projections, authenticated context, policy bytes and total deterministic authorizer yield identical decisions by substitution. Parity is therefore an expected result in this domain, not an unsuccessful security experiment. The converse is false: different projections may produce the same decision. The planted body-hash diagnostic gives a concrete counterexample.

I3: one gate-level send per reserved occurrence. Assume one intact shared local ledger, atomic exclusive creation, no occurrence deletion/reassignment and complete mediation through this gate. Only the caller creating a reservation can advance it toward dispatch, and that execution calls `createComment` once. Repeated occurrences do not send. This local at-most-one invocation property is not provider exactly-once behavior. A second host/directory, fresh occurrence for the same task, hidden adapter retries or provider duplication invalidates the premises or exceeds the conclusion. A crash before POST may also consume an occurrence without useful work.

I4: local freshness is not remote atomicity. Policy is loaded after awaited identity and preflight checks. Under trusted snapshot/policy ownership, a change completed before this load cannot be hidden by an earlier positive decision. A revocation after the load and before remote commit still defeats global atomicity. No additional application-level await is needed for that race; local sequencing does not create a distributed transaction.

I5: reconciliation creates no new write. The recovery routine prepares, loads the ledger and reads the provider. It has no create branch. With genuinely read-only provider methods it cannot resend. It also does not request a new write grant: observing prior delivery differs from authorizing another effect. Read authorization remains the caller/provider's responsibility. A malicious read adapter or operator rerun is outside the routine.

I6: two strengths of current confirmation. A known ID plus exact ID/API URL/issue URL/body/actor checks establishes matching current direct content. Without an ID, complete preflight and one exact match with a new ID establish new current content since preflight, not which execution caused it. A same-account writer posting identical bytes defeats attribution without defeating equality. This motivates the explicit ID-less qualifier *not execution attribution* rather than a stronger causal claim.

Control-flow arguments do not supply missing trust, completeness, currentness, durability or provider semantics. Set containment cannot establish those premises.

3 Relation to Existing Interfaces

3.1 MCP Tool Annotations

The selected `2025-11-25` schema includes read-only, destructive, idempotence, and open-world hints, with an explicit warning against trusting annotations from untrusted servers [S1]. It is not asserted to be the latest revision or the protocol actually negotiated in a run. A supported effect extension needs its own agreed semantics; there is no claim of an upstream-standard Action-Effect field. A capable MCP integration can already use a context-aware structured adapter, so hints-only comparison is insufficient.

3.2 AuthZEN and OAuth RAR

AuthZEN Authorization API 1.0 is Final, published 11 January 2026, and provides subject/action/resource/context decision exchange [S2]. Its successful decision is boolean with optional context. Review, unsupported semantics, and processing failure need explicit application-level mappings, not invented boolean values or automatic correctness credit for a failed PDP call.

RAR's structured `authorization_details` and separate objects preserve distinct permission combinations. RFC 9396 section 6.1 explains that arbitrary details lack a universal comparison operation independent of type semantics [S3]. Requested details, granted details, and described effects are different. Token privilege restriction, audience checks, and actual resource authorization remain necessary [S8]. Reusable evidence does not cache permission.

3.3 Complete Structured OPA and Cedar Adapters

OPA accepts complete structured input. Its Data API can return HTTP 200 without a `result` for an undefined document, while built-in errors ordinarily evaluate to undefined unless the selected strict-error mechanism changes that behavior [S4, S5]. The required total-decision wrapper must validate outputs and declare its error configuration identically across arms.

Cedar provides principal/action/resource/context requests, entities, policies, and schema validation. Its documented default-deny, forbid-overrides-permit, and skip-on-error semantics return diagnostics; an error in one policy can coexist with an Allow from another [S6, S7]. That is not automatically a Cedar defect. A stricter wrapper or review route must be explicit and symmetric. Policy validation does not authenticate facts. Cedar is a future comparator, not an executed lane of the current OPA pilot.

3.4 Provenance, Digests, and Provider Semantics

PROV-DM structures provenance but does not establish factual truth [S9]. JCS defines canonical JSON, not authority or semantic correctness [S10]. The finite consumer's numerical subset must not be presented as full binary64/JCS support. GitHub documents notification effects and secondary rate limiting for comment creation [S11]. Recipients, delivery, webhooks, and other descendants remain unresolved under the partial profile. Reading a comment does not establish their absence. HTTP retry semantics likewise do not make an uncertain create safe to repeat [S12]; a local marker or MCP hint is not a provider idempotency key.

3.5 Obligation-Level Comparison

The comparison unit is a boundary responsibility. A decision API need not infer effects, an effect schema need not issue tokens, and a policy engine need not inspect remote code. Table 4 makes the application obligations explicit instead of penalizing a component for another component's job.

Table 4: Primary-source obligation comparison. An application obligation is not a documented defect of the named interface.

Obligation	Existing facility	Integration boundary
Tool intent/shape	MCP tools, schemas, hints [S1]	Bind actual arguments and trustworthy semantics; no complete bound from hints
Detailed authority	OAuth RAR [S3]	Type-specific comparison, coupled permission objects and actual grant
Decision exchange	AuthZEN [S2]	Effect attributes and explicit error/review mapping
Structured facts	OPA/Rego [S4,S5]	Authenticate inputs, preserve tuples, validate total output
Typed policy	Cedar [S6,S7]	Explicit diagnostic and partial-scope handling; not measured here
Credential limits	OAuth BCP [S8]	Resource-server enforcement remains necessary
Artifact/derivation	JCS and PROV-DM [S9,S10]	Fixed preimages and trusted acquisition, not causal proof
Uncertain delivery	HTTP semantics [S12]	Occurrence ownership and no blind non-idempotent retry
Comment effects	GitHub REST/OpenAPI [S11,S13]	Exact body/issue and unresolved notification scope

The additional surface is agreement on description preimages, applicability, coverage, unknowns and rejection semantics. A direct adapter can retain those facts internally. The research question is whether portability reduces duplicated integration work or exposes errors across implementations. This pilot measures preservation and cost, not maintainer hours saved, and cannot isolate benefits of this wire format from typed contracts generally.

A stronger future baseline could export its own complete typed record under an equally explicit contract. It must not be deprived of hashes, context or failure reasons. If preservation remains equal, governance, migration burden and consumer availability may decide adoption rather than safety or speed.

4 Bounded Runtime Pilot

The published Spec prerelease `v0.2.0-draft.1` resolves to `a9b86f4`, pinned in Bench's `spec-lock.json`. The measured Bench source is `9f638df`. GitHub Issue Comment Profile 1 is additive on unchanged core wire version `0.1.0-draft.1`; its released source is `profiles/github-issue-comment/README.md`. Native SDK input contains owner, repository, issue number, and body; the MCP lane uses `tools/call` for `add_issue_comment`. Lane-specific capture envelopes are not a replacement MCP protocol. Equivalent intents compare checked projections, not full description digests that include different native lanes.

Reports record MCP SDK `1.32.1`, Octokit `22.0.1`, OPA `1.21.2`, Node `22.18.0`, and Python `3.9.6`, with source and binary hashes. Offline evaluation uses local SDK/MCP-shaped captures

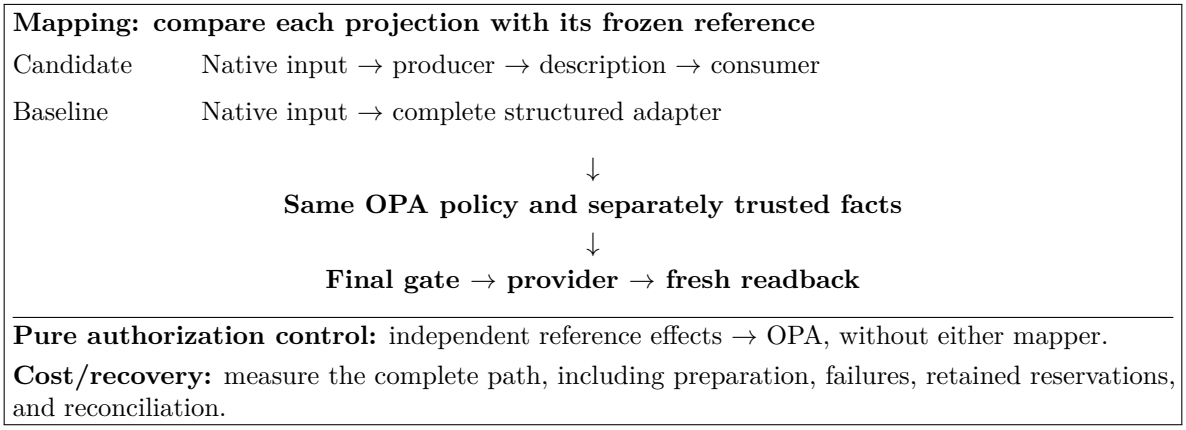


Figure 2: Measurement factorization. Offline scoring stops before dispatch; live execution also exercises the gate and provider. These stages have separate references. A correct-looking denial cannot conceal a wrong mapping, and a fault-induced stop is not semantic correctness.

and real OPA, not provider transport. The live MCP lane uses official SDK stdio initialize/list/call against the benchmark-authored `action-effect-comment-pilot` server, not GitHub's official MCP server. Both live lanes use Octokit against GitHub.

The profile binds the exact issue and destination to body SHA-256 and UTF-8 byte count. Its finite input domain includes a nonempty Unicode-scalar body of at most 8192 bytes. Data classification is trusted configuration, not DLP; `github-comment-bridge/1` is a checked declaration, not attestation of provider code. Success is `accepted_partial/declared_partial`, still without authority. After applicable denials, the experiment policy requires `allow_partial_direct: true`; otherwise partial scope requires review.

The protocol arm uses producer, description, separate consumer, real OPA, and a final gate. The main baseline substitutes a complete structured adapter but shares policy, authenticated facts, gate, occurrence ledger, observation rules, and offered work (Figure 2). A lossy-field ablation is not the main competitor. Only approved synthetic operations on owned public test issues may be dispatched; a stub or replay cannot count as live evidence.

Before POST, reserve the occurrence and recheck current local experimental authority and request bindings. This is not an atomic transaction with GitHub or instantaneous provider revocation. Once delivery might have begun, retain the reservation and reconcile read-only rather than automatically repeat POST. A single-host ledger does not establish distributed exactly-once execution. Known response IDs must bind the exact readback. ID-less recovery requires a complete pre-dispatch ID list, a previously absent ID, and a unique exact body/actor match on the dedicated single-writer issue. Its evidence explicitly means new current content, not execution attribution. A concurrent same-account writer violates that assumption. Missing or ambiguous evidence stays unknown.

4.1 Profile and Registry Lifecycle

The installed profile is a finite semantic program defined by a manifest and versioned rules. Repository release, core wire, processing contract, mapping revision, implementation declaration and executable versions are distinct. Release `v0.2.0-draft.1` did not change core `0.1.0-draft.1`; opt-in `offline-consumer-v1.1` clarifies legacy refund processing without rewriting its frozen contract or examples.

The comment manifest fixes lanes, operation, path-safe identities, limits, namespaces, local references, critical extension and unknown scope. A received URI cannot select arbitrary code. The Python consumer independently reconstructs the expected description instead of importing the producer or trusting its proposed projection.

The core requires immutable meanings and explicit experimental, stable, deprecated or withdrawn states. Changes to conditions or evidence rules require new identities. Withdrawal must not substitute bytes under an old revision: sensitive payloads may become unavailable with safe tombstones rather than remaining public just to preserve a digest. This is a lifecycle obligation, not an implemented signed registry or tested key-rotation service.

Cached bytes and cached acceptance are different. A document may remain readable while trust, context or evidence is stale. Acceptance reuse would bind full description, profile/mapping/checker revisions, implementation, trust epoch and dependencies. The pilot uses cold checking, not a persisted acceptance cache. Python memoizes only its installed profile within a process. Neither a record hash nor the occurrence ledger is permission reuse.

4.2 Ordered Native and Description Checking

Owner names are ASCII/path-safe, at most 39 characters. Repository names are path-safe, at most 100 characters, excluding dot and dot-dot segments. Issue numbers are positive integers at most 2147483647, not booleans or coerced strings. Bodies are nonempty Unicode-scalar strings of at most 8192 UTF-8 bytes. Whitespace and escaped NUL are preserved. Arbitrary endpoints, percent-decoding, templates and executable expressions are outside the profile.

Algorithm 1: Native Mapping

```
Map(N, C, P):
  require exact outer keys {lane, call}; lane is sdk or mcp
  if mcp:
    require {name, arguments}, scalar name, object arguments
    if name != add_issue_comment: unsupported_profile/operation
    A := call.arguments
  else: A := call
  require exact keys {owner, repo, issue_number, body}
  require identities, issue range and scalar/UTF-8 body bounds
  require principal, data_class, implementation in C
  require implementation == github-comment-bridge/1
  Q := complete seven-field projection
  D := Describe_P(N,C,Q), retaining the descendant unknown
  return accepted_partial/declared_partial, Q, D, H(D)
         authority:not_evaluated
```

Precedence is externally visible. Invalid native shape precedes context checks; another shaped MCP tool is unsupported without interpreting its arguments as a comment. Missing context and a different declared implementation have distinct untrusted-evidence reasons, not semantic review.

The Python request parser first bounds bytes/nesting and rejects malformed UTF-8, duplicate members and JSON framing. It represents fraction/exponent or oversized native numbers outside the accepted domain without rounding or arbitrarily large integer allocation. Native validation classifies those and native scalar defects as `native_input`. Accepted inputs are reparsed from the original bytes under the strict core parser before hashing. Preliminary values never become digest preimages. This preserves the frozen core rules while repairing the first run's native-error precedence

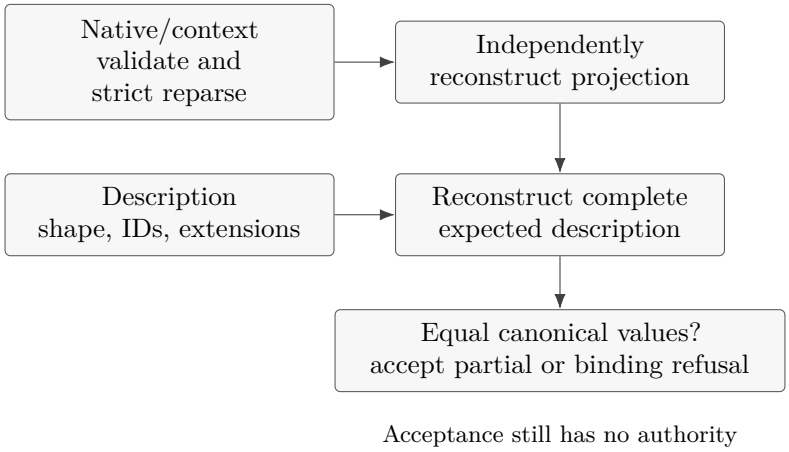


Figure 3: Ordered checking reconstructs meaning from bound inputs. A producer's projection or self-declared verification flag is not the answer key.

mismatch.

Algorithm 2: Independent Consumer

```
Check(frame):
  bounded request parse; require project/check and allowed keys
  load configured profile; initialization failure is explicit
  validate native input and context
  reparse original bytes with strict core parser
  Q := independently Project_P(N,C)
  if project: return accepted_partial, Q, authority:not_evaluated
  validate D's core structural shape
  resolve unique effect/evidence IDs and quantity targets
  reject unknown critical semantics; require body-hash payload
  require every necessary byte-quantity representation
  E := independently Describe_P(N,C,Q)
  if J(D) != J(E): untrusted_evidence/description_binding
  return accepted_partial/declared_partial, Q, H(D)
         authority:not_evaluated
```

Full comparison rejects altered coverage, assumptions, extra/missing evidence and reordered arrays, not just wrong disconnected digests. Its strength is also a limit: equivalent meaning in an unsupported representation is not accepted. This is finite-profile consistency, not general semantic equivalence. Limit or worker failures cannot become truncated accepted output.

4.3 The Exact Complete Baseline

Protocol preparation uses JavaScript `mapNative`, a full description, and Python `check`. Structured preparation directly invokes Python `project`. Both return operation, repository, issue number, endpoint, body hash, byte count and class, with the same declared-partial status. The baseline has no deliberately missing tuple, digest or unknown information.

The checker and baseline share Python's projection logic; the JavaScript producer was separately implemented. This isolates wire construction/validation while retaining shared-code correlation. These are not independently controlled end-to-end stacks or an external product comparison. Literal

Obligation	Protocol	Complete structured
Native/context validation	JS precheck and Python	Python
Complete projection	Producer proposes; checker reconstructs	Construct directly
Wire/evidence association	Full description and exact check	No effect-wire round trip
Python boundary	Cold check process	Cold project process
Authority	Same wrapper, executable, Rego, policy	Same
Principal/coupled grants	Same authenticated facts	Same
Reservation/dispatch	Same gate, ledger, provider	Same
Readback/recovery	Same exact checks, no resend	Same

Table 5: Matched protection and intentionally different preparation. Common gate protections are shared by both strategies.

references and pure authorization checks address different errors, not every shared assumption.

The authority wrapper validates policy, projection, context and local OPA configuration in that order. It reconstructs allowlisted input rather than serializing arbitrary objects containing hooks or secrets. A bounded OPA process evaluates `data.action_effect.decision`. Exactly one expression must contain a recognized decision/reason pair. Missing, malformed or failed output is failure; process success alone is insufficient authority.

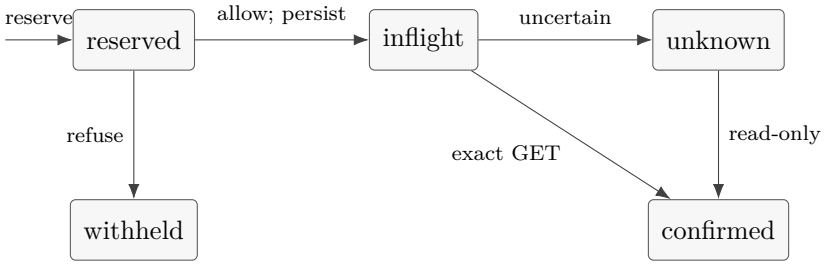
After validation, the decision order is revocation, principal mismatch, absent coupled grant, body-byte excess, unaccepted partial scope, then allow. The first four are denials and the fifth is review. The body hash is structurally checked but policy has no independent expected-body value. Correct binding belongs to the mapping/checking/gate path. This explains diagnostic masking without implying an OPA defect.

4.4 Dispatch State and Failure Cuts

Before its first await, the gate snapshots native/context/optional description, strategy, policy/ledger paths, occurrence and bound provider methods. JSON is canonicalized, strictly reparsed and recursively frozen. Caller mutation across identity lookup or preflight cannot replace these options. The technique does not freeze remote state or establish trust in adapter internals.

Preparation precedes exclusive reservation. A matching existing occurrence is `duplicate_suppressed/already_reserved`; a different binding is suppressed with `binding_mismatch`. Neither creates fresh authority. For a new reservation, authenticated-principal comparison and complete comment listing precede policy loading. Existing matching body/actor content withholds. Only allow can persist inflight with pre-dispatch IDs and then invoke the write.

Reservation uses exclusive creation, restrictive permissions, file fsync and directory fsync. Transition obtains a record lock directory, verifies the expected state, writes/syncs a unique temporary file, renames it and syncs the directory. Metadata is allowlisted; stale locks are not stolen. Failure may retain reserved/inflight state and require intervention. Conservative retention has a liveness cost and cannot count as useful completion.



Existing occurrence: suppress new send
No retry-to-write edge from unknown or terminal states

Figure 4: Local ledger transitions, not a remote transaction. A crash can leave an earlier durable state; an unknown does not release occurrence ownership.

Table 6: Failure cuts from control-flow analysis. The v0.2 live run did not exercise every crash cut.

Cut	Information	Permitted continuation
Before reservation	No gate-owned occurrence	Repair preparation; provider not reached
Reserved, before preflight	Record exists; no send edge	Suppress repeated occurrence; account for lost work
Preflight, before policy	Current IDs available	Load current policy; failure withholds
Allow, before inflight	Local decision only	Failed persistence must not send
Inflight, before call	State cannot distinguish a later crash	Read-only recovery, no resend
Commit, absent ACK	Object may exist, local ID absent	Preflight-qualified readback or unknown
Response ID, failed GET	Possible commit with ID	Read exact ID, not a substitute
GET succeeds, persistence fails	Observation ahead of durable state	Reconcile/read, never another write
Lock survives exit	Lock ownership unresolved	Withhold; operator handling

4.5 Bound Observation and Recovery

`receiptFrom` checks a positive safe-integer ID, exact expected ID when available, exact API/issue URLs, exact body and exact actor. Hash and byte count summarize content; a display URL or timestamp cannot replace these predicates. Provider calls use constructed GitHub requests, redirect refusal and bounded timeouts. Credentials never come from a proposal.

Listing reads at most five pages of 100 and succeeds only after a shorter page. Five full pages cannot establish completeness and throw even if exactly 500 comments might exist. Absence on page one is not absence from the issue. Duplicate, invalid or oversized ID lists are rejected before storage.

Reconciliation loads the original occurrence and checks its native/context binding, proceeding

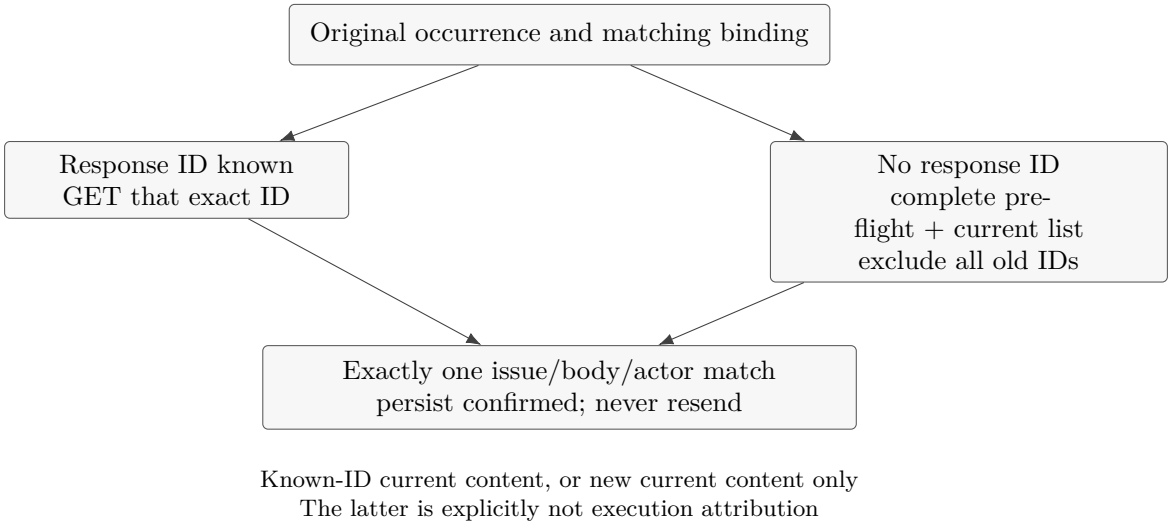


Figure 5: Two observation strengths with one no-resend rule. A trusted successful-ACK-discard witness is separate from recovery's evidence.

only from inflight or unknown. A response ID selects exactly that object. Without an ID, persisted preflight is required, every old ID is excluded, and exactly one matching body/actor must remain. Missing/multiple matches, unavailable pages, changed bindings and failed persistence remain unresolved. An old comment edited to match cannot qualify as new content.

The state word confirmed must be read with its qualifier. The ID-less evidence is `new-current-content-since-preflight-not-execution-attribution`. A dedicated single-writer issue limits ambiguity, but a same-account concurrent writer defeats attribution. The observer reports checked current content, not universal exactly-once or comprehensive effects.

5 Falsifiable Questions and Experimental Design

`PROTOCOL.md` governs the study; this paper does not silently change its corpora, labels, scoring, or independence requirements.

1. **Meaning preservation:** do both arms match reference coupled effects, binding state, and uncertainty across equivalent intents? Baseline parity supplies no added mapping benefit; extra loss or unsupported outcomes count against the proposed interchange.

2. **Diagnostic value:** can separate mapping, reference-input authorization, and integrated scores reveal policy-masked mapping defects? Report their joint correctness cells, with additional explicit failure cells. A complete baseline may expose the same distinctions.

3. **Unknowns and freshness:** do prewritten drift, authority-change, capability, and delivery cases preserve required limits? Fabricated known facts, stale acceptance, error-as-review, and permission reuse contradict the contract.

4. **Cost and utility:** does any demonstrated portability or protection benefit justify complete cost, withheld work, intervention, and recovery? Negative findings and a better baseline remain valid outcomes.

5.1 Arms and Ablations

Both full preparations were measured with the same OPA and gate. The lossy-body projection is a separate diagnostic, not a competing system. Further binding, unknown/failure or invalidation ablations and a typed Cedar adapter remain proposals rather than executed lanes. Pure authorization uses case-author reference projections without either candidate mapper. A broken baseline must be repaired and its history retained, not interpreted as semantic superiority.

5.2 Unit, Freeze, and Exposure

The base intent is the sampling unit; variants, repeats, languages, and lanes are not extra independent cases. This is an author-designed convenience pilot, not a customer-workflow population sample. Competence and tuning budgets must be matched in public development. An OPA evaluator cannot define the expected answers and then receive independent correctness credit for matching itself. Pure authorization uses separately authored reference effects and decisions.

Authoring and independence. Implementation, separate case authoring, and review were AI-assisted within one project. Literal references were authored without candidate-output inspection; implementation proceeded in parallel from the agreed public interface. The corrected custody record describes withholding cases until a *pre-execution/pre-result* freeze, not a pre-implementation freeze. Subsequent failures became exposed regressions. The editorial audit checks retained reports after execution, not independent reference authorship. No external evaluation or protected external cohort occurred.

The selected official OpenAPI commit is `755ee33b4a81d51f6f07a902cce26cbec3b822f2` [S13]. The pinned excerpt retains its MIT notice and supplies structural examples and notification wording, not independent governance labels. Official commit identity and local extraction hashes remain separate records. After a learned failure, that family becomes exposed development; hiding or hashing it does not restore unseen status.

Protected external evaluation additionally requires independently authored inputs and outcomes, separate sealed access, actual custodians, independent disputed-score governance, and disclosed conflicts. System exposure is not developer access. Hosted-provider logging can undermine secrecy and must be documented. Two languages, repositories, or AI-assisted sessions do not establish independent laboratories. Active protected traces and labels stay sealed until retirement and rights review.

5.3 Pipeline-Masking Diagnostic

Each eligible input has two correctness indicators: mapping and decision. Their four joint cells are correct/correct, correct/incorrect, incorrect/correct and incorrect/incorrect. Processing and observation failures remain additional states, not forced into a binary success table. A wrong mapping with a correct-looking downstream denial belongs to the third cell. It is not successful mapping.

A mapper or PDP crash followed by withholding likewise is operational non-execution, not correct policy recognition. A complete baseline can expose the same distinctions. Whether standardized interchange makes them easier to reproduce across implementations remains a testable engineering question.

5.4 Real Maintainer Use as a Separate Study

The approved field-use design extends the question beyond scripted comments. It separates ordinary model-driven maintenance, native integration conformance and induced faults. Old scripted slots cannot be renamed autonomous useful tasks. Two executions used the same six real read-only maintenance goals (Sections 8.9 and 8.10). The second is a post-review repeat under an amended output contract, not six new independent intents. Neither field run executed the larger design's new write or delivery-fault populations.

Ordinary tasks inspect actual public release material and produce maintainer reports about source/release binding, installation, contract migration, native compatibility, recovery limits and documentation. Goals, bounded tools, source snapshots, budgets, completion checks and failure rules are fixed before scored runs. The model chooses reads and text. Canned answers, reference answers in tool data, or changing expectations after seeing outputs defeat that design.

Read-only tools are constrained to explicit repository/ref/path bounds rather than arbitrary shell or URL access. That is a study capability boundary outside the comment-mutation profile, not coverage of reads by that profile. Repository content and tool descriptions remain untrusted data. Each captured write proposal can receive both preparations, with one authorized execution selected to avoid duplicating public work. Two shadow decisions are not two provider trials or a paired end-to-end model experiment.

Provider mutation is a separate authorization boundary, not a prerequisite for report-writing tasks. Without a separately confirmed fresh owned resource, the study can complete read-only reports and shadow qualification of captured proposals. That mode cannot establish official-server comment delivery, mutation recovery, or end-to-end behavior of the new write integration.

Native integration must distinguish the benchmark-authored MCP server using the official SDK from GitHub's official implementation. The latter requires its own executable/source pin, advertised schemas and strict result decoding. Remote authentication failure remains a setup result. A pinned official local executable may be an alternative, not evidence that the remote endpoint worked. Neither version strings nor executable hashes establish complete effects.

Induced faults separately schedule process exits, disconnected transport, policy narrowing/revocation, duplicate occurrences and ambiguous ACKs. After delivery might have started, continuation is read-only. Unfinished tasks, stale locks, unknowns, setup failures and interventions remain visible. Writes stop for unexpected targets, drift, missing journals, exhausted budgets or unexplained outcomes. Model safety stops are respected rather than bypassed through another interface. The read-only runs do not measure this write/fault boundary; Section 8.12 reports the separately frozen continuation.

5.5 Official-Server Compatibility Boundary

The field integration selects GitHub MCP server v2.0.2, source `eb47a99`. The selected Darwin ARM64 archive has SHA-256 `5d9b38511bffc84d2272eb6189129eb721c8900e45784d8ea7eaf25f8d3dd835`. The successful installation record verifies it and a distinct executable hash, `2d431e46617f06b151 6818fa09917096ff6dfc3664b37b0bdb31e1c1a35be368`. These pins identify the local artifact, not every acquisition attempt or hosted instance, and do not establish a reproducible build from the pinned source. They differ from the old handmade server and SDK package.

The pinned official typed-tool schema documentation [S15] distinguishes legacy output from modern `2026-07-28` typed output. SDK 1.32.1 negotiates no later than `2025-11-25`; the actual

Dimension	Old measured lane	Field configuration
Server	Benchmark-authored	Pinned official GitHub implementation
Client	SDK 1.32.1	Respect its negotiated-version ceiling
Result	Instrumentation envelope	Versioned upstream result, strict decode
Scope	Exact comment operation	Reject reaction variants
Delivery	Common gate/readback	One application call is not one wire attempt
Task unit	Scripted cases/slots	Same six goals in two runs; three SDK, three MCP each

Table 7: The new integration is not retroactive official-server coverage for the old experiment. Configuration does not establish task success.

negotiated revision must determine decoding. Legacy comment results contain JSON text with `id,url`; modern typed results use `id,html_url` with a string ID. Bridging to the old gate's safe-integer ID contract requires explicit decimal syntax and range validation, not coercion or opportunistic shape acceptance.

The pinned official issue-tool handler [S16] also supports reactions. This closed profile does not: reaction variants and extra mutation arguments must be rejected, not stripped to force a match. One handler-level `CreateComment` call is not proof of one HTTP attempt; Go redirect/transport behavior is another layer. Reviewing local source does not establish hosted-server behavior. The official server configuration [S14] is part of the integration contract, not its measured outcome.

The existing Azure deployment returns model ID `gpt-6-astra`; its configuration declares version `2026-09-03` and region Sweden Central, using Responses v1. These are not an immutable model-weight identity. Section 8.8 reports a connectivity response, not a scored maintainer task. Read tools remain operator-allowlisted capabilities outside comment-profile semantics. Six goals are not incidents; different SDK/MCP task goals do not support paired end-to-end speed comparisons.

Microsoft documents the resource-specific `/openai/v1/responses` endpoint [S17]. In the implemented and recorded field loop, a model `function_call` requests application work; a matching `function_call_output` returns it under the same `call_id`. The harness validates names/arguments against fixed read interfaces before calling GitHub. Requests use `store: false`, low reasoning effort, non-streaming foreground execution and no parallel tool calls. Limits are 6000 output tokens, 180 seconds/request, 12 rounds/task and 18 tool calls/task. These are distinct from the report's UTF-8 byte budget. Loop settings and deployment observations come from the retained implementation and run, not a general availability catalog.

6 Metrics and Analysis

Primary rates use their full frozen scheduled populations, including unsupported, missing, and failed output. Successful-only rates are supplementary. Pure malformed-wire cases occupy a predeclared separate track; no favorable post hoc exclusion is allowed.

Endpoint	Numerator	Denominator
False allow on block	Valid semantic allow in B	All B
False allow on review	Valid semantic allow in R	All R
False block	Valid semantic block in A	All A
Correct review	Valid semantic review in R	All R
Unnecessary review	Valid semantic review in A	All A
Legitimate work withheld	Any non-dispatch in A, including faults	All A
Forbidden dispatch	Dispatch contrary to the reference	All expected withhold

Table 8: Frozen-population endpoint definitions. A fault-induced stop is failure and possible utility loss, not correct semantic block/review. An all-block system cannot win through a single false-allow statistic.

6.1 Mapping, Unknowns, and Failure

Report exact semantic match, coupled-tuple and per-field loss, correct scope, modality, and bindings, together with coverage and reference ambiguity. For reference-unknown slots, report correct unknown retention and unjustified known/none assertions; for reference-known slots, report avoidable unknowns. Missing output earns no unknown-retention credit. Capability/processing checks may correctly return unsupported without establishing successful effect mapping.

6.2 Authorization and Admission

Let A, B, and R be the preassigned expected-allow, expected-block, and expected-review populations. Table 8 defines rates, not results. Retain full confusion tables including non-decisions for pure authorization and integrated admission. Record the gate and actual dispatch independently.

6.3 Completion, Recovery, and Observation

Separate invalid input/configuration/output, unsupported semantics, untrusted evidence, timeout, crash, missing output, and policy/provider/observer failures. Retain every attempt and link each permitted rerun; do not select the best run. Intentional non-dispatch is not attempted delivery. Provider outcomes can be confirmed, partial, unknown, or conflicting. Unknown-delivery rates use all attempted occurrences, also showing counts against scheduled work.

Original task completion uses all scheduled legitimate tasks; withholding, timeout, abandonment, and missing completion are non-completions. Report forbidden effects, duplicates, retained reservations, reconciliation, and human burden. Joint verified useful-and-safe completion needs the original goal, no forbidden effect in the declared universe/window, and sufficient observation. Unknown coverage cannot qualify. Direct-comment-only success must be called scoped verified completion, not safety of every descendant. Separate clean and challenged workloads and label conservative sensitivity bounds as bounds.

6.4 Complete Cost and Statistical Limits

Retain raw monotonic timings for preparation, parsing, mapping, serialization, validation, evidence checking, invalidation, authorization, final recheck, reservation, transport, provider response, readback, and reconciliation. Include end-to-end boundaries, cold/warm state, startup, offered/rejected

work, input size, load, resources, network, provider calls, and measured human or monetary cost. Do not sum overlapping timers or turn unmeasured cost into zero.

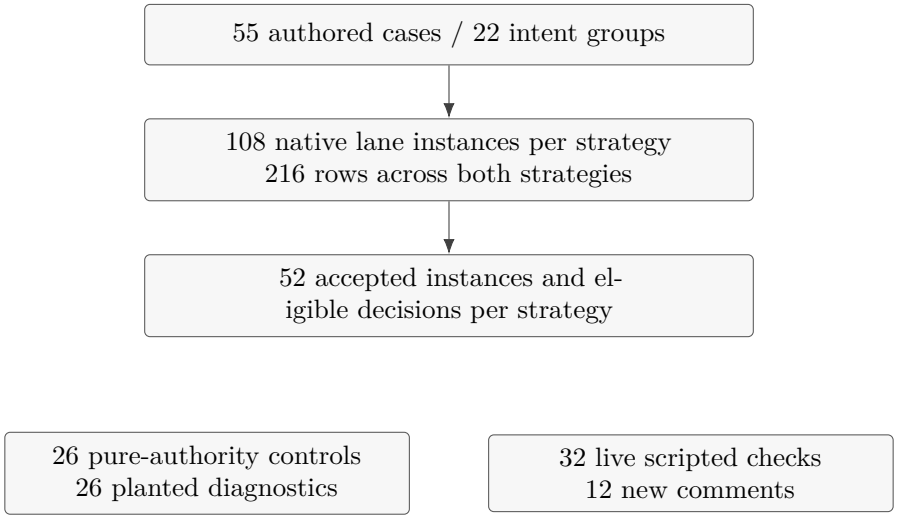
Use paired intent-level comparisons and predeclared family weighting, clustering variants and repeats. Small convenience samples warrant descriptive findings, not population coverage or unsupported tail-SLA claims. Report sample support for percentiles. Zero observed failures is not zero risk; correlated variants do not justify an independent-trial bound. Preserved utility or review burden needs justified preregistered non-inferiority criteria, not a nonsignificant difference. Publish negative results and analysis deviations.

6.5 Measurement-to-Claim Map

A claim must identify its population, reference and observation boundary. Mapping exactness includes expected nonacceptance; decision accuracy applies where a decision reference exists. A native lane is not an intent, a test method is not a case, and a provider object is not a completed useful task.

Table 9: Traceability from measurement to bounded claim.

Claim	Unit/reference	Evidence	Not established
Mapping parity	108 lane instances/arm; literal labels	Status, reason, projection equality	Arbitrary inference
Decision agreement	52 accepted instances/arm	Real OPA and frozen decisions	Mapper truth from decisions
Pure authority	26 reference projections	No candidate mapper input	External oracle
Masking	26 corrupt projections	Zero exact, 26 unchanged decisions	Attack prevalence
Live behavior	32 scripted slots	Exact reasons/outcomes/witnesses	32 useful tasks
Direct content	12 unique objects	Bound fresh readbacks	All descendants
No recovery resend	Four injected ACK losses	Recovery/send accounting	Distributed exactly-once
Accepted cold cost	52 rows/arm	Monotonic row timings	Task speedup
Ordinary live cost	Four rows/arm	Sequential process/HTTP time	Ranking or SLA
First-run reports	Six real-model tasks	Six mechanical passes; zero delivery-compliant responses	Six useful reports
Amended repeat	Same six goals, fresh contexts	Two completed, four retained byte-budget failures	Causal effect or twelve independent intents
Content review	Thirty fixed facts/run, same-project reviewer	First: 27 met/2 partial/1 incorrect; repeat: 28/2/0	Independent truth or human acceptance



Separate study: six read-only goals in two runs; 24 packaging rows per run
Second-run cohort: two eligible / four task-layer withheld
Two ordinary deliveries + six fault scenarios = eight initial proposals
Eight initial + four read-only recovery + eight duplicate probes = twenty workers
Five calls / five objects: two reports and three fault controls
Paired initial preparation: eight per strategy, not sixteen live trials

Figure 6: Denominator map. Rendering and repetition do not create new independent intents; conformance and task utility cannot be pooled.

Claim	Unit/reference	Evidence	Not established
Publication fit	Each run's six outputs, two forms/strategies	24 related rows/run; equal categories	Task success, policy or delivery
Owned delivery	Two eligible outputs from six candidates	SDK and official-MCP publication with readback	Six successes or human utility
Owned faults	Six conditions, four recovery processes	Two denials, three recovered, one unknown	Natural outage rate or atomicity
Duplicate suppression	Eight fresh-process probes	No further native comment calls	Exactly-once across lost ledgers
Task utility	Original goals and criteria	Internal rubric; human use remains unknown	Correctness from allow
Reproducibility	Reports, inventory, journal	Local replay/current GETs	Historical causality

For task runs let T be scheduled legitimate tasks, U correctly completed useful tasks and V those

with adequate evidence of permitted effects in the declared universe. Report $|U|/|T|$ separately from $|U \cap V|/|T|$. If k completed tasks lack sufficient effect observation, the range from known verified tasks to those tasks plus k, divided by $|T|$, is a missingness sensitivity range, not a confidence interval or extra observed successes.

Cost counts model requests, tool attempts, supported write proposals, POSTs, GETs and intervention separately. Provider-reported tokens are not invoice cost. Elapsed time is not the sum of overlapping spans. Setup failure before tasks is not task failure, but cannot be removed from the engineering account.

7 Scope and Validity

Semantic coverage. The mutation profile covers one direct GitHub comment. Notifications, recipients, webhooks and other descendants remain unknown; accepted mappings are partial. The original live data were synthetic public inputs. Real-model maintenance adds bounded reading/report generation, not profile-governed arbitrary reads. Publication-fit preparation does not dispatch; the distinct owned-write continuation in Section 8.12 observes official-server mutations through the same finite direct-comment profile. Its deliberate faults are not naturally observed outages.

Trust and concurrency. The host, configured principal/class, provider adapter, policy files and same-host ledger are trusted. Bridge identity is declared, not SaaS attestation. Invariants are code-level arguments, not machine-checked whole-system proofs. Policy reload is not atomic with remote commit. ID-less recovery requires complete preflight, unique new content and a dedicated single writer; another same-account writer defeats attribution. Hashes and replay establish specified consistency, not operator honesty, historical fsync or distributed exactly-once delivery.

Sampling and independence. Shared policy, contracts, project control and AI-assisted authoring/review permit correlated errors. Separate consumers and references are checks, not independent institutions. External independently authored evaluations number zero. The six maintenance goals use the project's exposed public sources; post-review repeats remain exposed follow-up work, not holdouts. A rubric requesting human review does not establish that it occurred; actual review provenance is reported.

Inference and deployment. Small sequential cold timings support their samples, not throughput, tail-SLA, staffing or monetary claims. Different SDK/MCP task goals are not paired speed observations. Loop completion, citation availability, quality, decision readiness, actual use and delivery are distinct endpoints. Related work compares concrete interface obligations rather than surveying all effect systems or authorization theory. Customer qualification and long-duration benefit need additional evidence.

8 Results

8.1 Sources and Denominators

The earlier refund rerun matched 102/102 expected outputs per consumer with no adapter faults. Opt-in `offline-consumer-v1.1` clarifies startup and compound-error precedence without rewriting

frozen v1. At measured v0.2 source, Linux/macOS CI passed 64 Spec JavaScript, 11 legacy Bench JavaScript, 115 Spec Python, 333 runtime JavaScript and 50 runtime Python methods: 573 methods, not workloads. Eight gate regressions concerning stale content, ID binding, mutable configuration and policy freshness were repaired before that source. These are retained development history, not another independent cohort.

The offline and live reports use the linked Bench and Spec commits above and the same 49-entry source inventory; each entry matches its frozen Git blob in the editorial audit. The recorded Bench checkout is dirty, including reports and documentation, not represented as clean. Exact filenames, hashes and commands are in `measurement-contract.md` and `sources.json`.

There are 55 authored cases in 22 base-intent groups, with 53 SDK and 55 MCP inputs: 108 lane instances per strategy and 216 strategy rows. Two omitted SDK renderings are inapplicable MCP-specific inputs, not failures. Accepted input comprises 26 authored cases in 13 groups, rendered in both lanes: 52 rows per strategy. These related observations do not support independent-trial or population-accuracy claims.

8.2 Offline Mapping and Authorization

Check, per strategy	Protocol	Structured
Exact status, reason and supported projection	108/108	108/108
Accepted partial mappings	52	52
Invalid / untrusted / unsupported	51 / 4 / 1	51 / 4 / 1
Exact eligible decisions	52/52	52/52
Allow / deny / review	22 / 28 / 2	22 / 28 / 2
Authorization not evaluated	56	56
False allows on deny/review	0/30	0/30
Expected allows not allowed	0/22	0/22
Real masked errors / operational failures	0 / 0	0 / 0

Pure authorization matched 26/26 separately authored reference decisions: 11 allow, 14 deny, and one review. These controls share the authored cases; they are not an additional independent cohort. Mapping nonacceptance is not credited as policy denial.

A deliberately lossy diagnostic replaced 26 reference body digests with zeros: mapping matched 0/26 while policy decisions matched 26/26. This illustrates why decision accuracy cannot substitute for mapping checks. It is not a competitor weakness, empirical attack rate, or naturally observed error in either complete strategy.

8.3 Controlled Live Outcomes

The run on issue #1 lasted from 04:24:32.195Z to 04:26:11.245Z on 19 October 2026. Eight scripted scenarios crossed with two strategies and two lanes yield 32 scheduled checks, not 32 independent incidents or successful creation tasks.

Observed category, per strategy	Protocol	Structured
Planned outcome/reason checks	16/16	16/16
Ordinary immediate confirmations	4	4
Lost ACK: unknown then read-only confirmed	2	2
Duplicate occurrences suppressed	2	2
Withheld: deny / review / invalid-policy failure	4 / 2 / 2	4 / 2 / 2

The issue started empty and ended with 12 run-scoped new comments: eight normal writes and four controlled lost-ACK writes. Duplicate occurrences added none. The four initial unknowns are 4/12 new-create attempts and 4/32 scheduled checks by design, not an outage estimate. Each has a `successful_post_ack_discarded` witness and read-only recovery with `automatic_resend:false`. The injected loss follows a real successful POST response, not a naturally occurring outage. Recovery establishes new current content, not general execution attribution.

Four invalid-policy rows are evaluation failures, correctly withheld, not semantic denials. Thus 32/32 planned checks does not mean every row succeeded operationally. Direct-content confirmation leaves downstream effects unknown.

8.4 Cost Without a Speed Ranking

Median elapsed time, ms	Protocol	Structured	n/path
Accepted offline input	50.391	48.468	52
All offline input	0.042	44.507	108
Ordinary live completion	3110.848	3161.205	4

Accepted-input median cost is 1.923 ms, or **3.97% higher**, for the protocol. This is a ratio of marginal medians, not a median paired percentage or significance claim. Accepted includes deny/review: it is not provider-task latency. The all-input median crosses a different boundary: 56 protocol rows stop in JavaScript without Python startup, while the direct adapter starts Python. It is not comparable useful-task speedup.

Accepted offline cost includes cold Python communication/checking and real OPA startup. Live cost also includes identity lookup, preflight, ledger, POST and readback, plus cold MCP transport where applicable. The entire run took 99.049 seconds, including one-second pacing waits excluded from row timers. Initial live rows overlapped same-host read-only offline replay; the host was not isolated. Four normal observations per path support neither speed ranking nor SLA. The host reports Darwin arm64, Apple M4 Pro, 14 logical CPUs and 24 GiB memory; Toronto is editorial metadata, not geolocation.

8.5 Retained Failure and Replay

The first failed offline report is retained. Its scorer reported 52 apparent masked errors per strategy from null-prototype versus ordinary-object comparison. A JSON-value audit finds 108 exact protocol and 100 exact structured mappings; eight structured numeric/Unicode error-classification mismatches remain. These are not 52 real policy-masked defects. All 216 input/context/ policy and expected records are unchanged in the final report. Repairs changed code and scoring, not labels. The complete initial executable tree was not archived; this is retained evidence, not exact old-source reproduction.

Offline replay passed 216 strategy rows plus 52 reference/diagnostic records. Live replay passed 32 rows and matched all 12 unique comments using anonymous GETs at its recorded check time. The attempt journal has one begin, 32 scheduled, 32 result, and one end record consistent with the live report. Replay verifies recorded consistency, calculations, and current content, not operator honesty, historical latency, fsync, or complete causality. Later unavailable, changed, or rate-limited content cannot be treated as a match. The full artifact map is in `measurement-contract.md`.

8.6 Case Analyses: What Aggregates Hide

The Unicode cases test byte semantics, not natural-language correctness. `unicode-nfc` and `unicode-nfd` preserve distinct precomposed and decomposed bytes. A four-character cafe spelling with an accented final letter occupies five UTF-8 bytes: a five-byte policy limit permits it and a four-byte limit denies it. The native cap instead accepts 8192 bytes and rejects 8193 before policy. These are different obligations; a native-domain error is not credited as a correct current-policy denial.

The tuple cases distinguish cross-repository/issue and cross-class recombination from an exact grant among decoys. The first two deny despite individual field matches; the third allows because one whole row matches. A deny-everything system fails the positive control. Case-sensitive owner/repository variants are mapped literally and denied against differently cased grants, not normalized through guessed provider aliases.

Precedence cases combine revocation, absent grants or excessive bytes with unaccepted partial scope. Known denials win over review. The otherwise permitted partial-scope case still receives review, while missing context or a wrong implementation produces untrusted evidence before authorization. These tests check refusal ownership, not simply non-execution.

The first scorer defect compared language-object prototypes instead of JSON values and reported 52 apparent masked errors per arm. Correcting comparison exposed eight genuine structured numeric/Unicode classification mismatches. Those mattered to interoperability even though they did not grant unsafe work. All 216 original input/context/policy/expected records remained unchanged. The lesson is to specify comparison and precedence, not erase failure history or claim repair of 52 dangerous defects.

The live duplicate reused the earlier ASCII occurrence: its suppression is not a second denial or an extra object. The ACK-loss scenario first completed a real POST and deliberately discarded its successful response. Recovery therefore has a controlled injection witness as well as weaker current-content evidence. Conflating those would generalize the result to arbitrary unwitnessed timeouts.

8.7 Cost Decomposition and Identifiable Contrasts

Let T_P, T_S be protocol and structured row times. The accepted contrast includes launch, Python parsing/projection, optional description construction/checking and OPA startup/evaluation. Difference of medians is not median paired difference, nor an isolated serialization constant. Cold-process results do not identify warm-service throughput or a cache break-even point.

The data contain no maintainer-hour measurement, price schedule, sustained load sweep, randomized-order experiment or demonstrated amortization across consumers. Portability must pay for schema/registry maintenance and triage as well as per-call checking. The 3.97% higher accepted median is the bounded finding, not a concealed performance advantage.

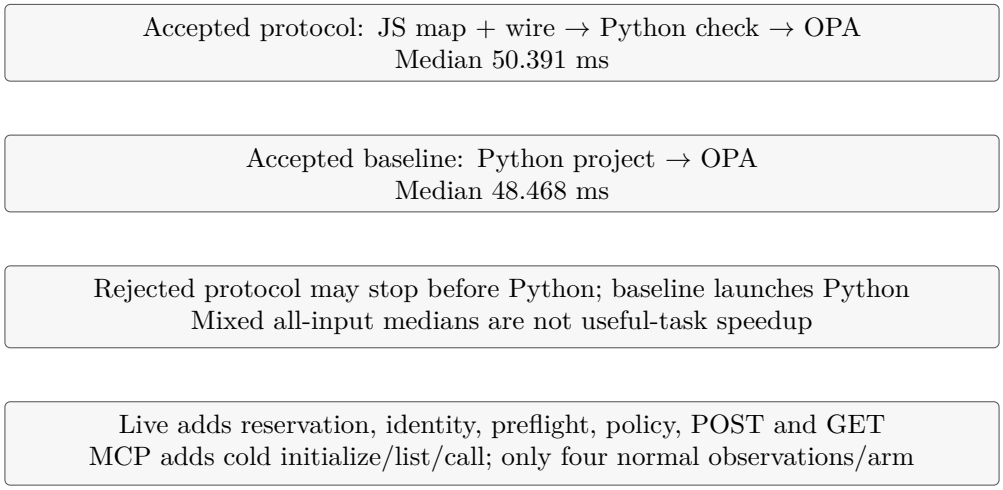


Figure 7: Inclusion boundaries, not additive stacked timing. Mapping and consumer timers overlap. Pacing lies outside row timers and inside entire-run time.

Setup observation	Elapsed time	Outcome and boundary
Initial archive acquisition	91 s	curl HTTP/2 framing failure, exit 16
Hostname-only MCP flag	235.459 ms	Real parser rejects missing HTTP(S) scheme; no tool dispatch
Corrected MCP preflight	924.395 ms	Negotiated 2025-11-25; read pinned Spec lock, no mutation
Azure connectivity	4813.349 ms	Received READY; setup includes credential retrieval

Table 10: Observed setup events, not scored maintainer tasks. Acquisition time has only the coarse precision reported in its retained record.

8.8 Field Setup and Native Compatibility Observations

Setup is its own population, `setup-not-task`. Failures and successful connectivity are not reassigned to the six prospective maintainer tasks. Every report below records zero provider-write attempts. Times are individual setup observations, not throughput measurements or task-level model latency.

The initial report `field-study-setup-2026-10-19T06-06-29.292Z-83109149.json` in the reports directory retains the 91-second failure. The subsequent HTTP/1.1 installation record, `field-study-setup-2026-10-19T06-10-02.910Z-8154f464.json`, verifies both artifact hashes in Section 5.5. Retry success does not erase acquisition cost or establish a task outcome.

The first MCP preflight, `field-study-preflight-2026-10-19T06-14-44.920Z-85a0901c.json`, records `mcp_initialization_failed` and `not_dispatched`. Its diagnosis, `field-study-setup-2026-10-19T06-15-33.486Z-132f0b63.json`, preserves the real parser's rejection of `-gh-host=github.com`. Fixtures had accepted that hostname-only configuration. Correcting it to `-gh-host=https://github.com` and adding a regression exposes an integration gap mock compatibility did not detect, not a mapping failure or false authorization decision.

The corrected read-only preflight, `field-study-preflight-2026-10-19T06-17-11.024Z-4a25c7d7.json`, records server 2.0.2, protocol `2025-11-25` and allowlisted tools `get_file_contents`, `get_release_by_tag` and `issue_read`. The actual call read `spec-lock.json` at Bench `cecde38`,

Task	Lane	Model	Reads	Wall, ms	In tok.	Out tok.	Bytes
T1 Release integrity	SDK	7	6	79893.440	24207	2132	5872
T2 Clean-clone guide	MCP	6	5	50447.211	25493	2201	7270
T3 v1.1 migration	SDK	4	3	44750.881	17145	2128	7981
T4 MCP compatibility	MCP	6	5	49512.368	22108	2073	7284
T5 Limits/recovery	SDK	6	5	50971.146	33098	2050	6836
T6 Conformance	MCP	6	5	51169.527	33292	1956	7159

Table 11: First-run task observations. MCP uses the official GitHub server. Model counts requests; bytes measure the report body before publication packaging. Different goals and read sequences prevent lane-speed comparison.

whose resource content contains Spec pin `a9b86f4`. An advertised read tool is not automatically tested; this does not exercise the official comment handler or its response decoder.

The model preflight, `field-study-preflight-2026-10-19T06-15-46.567Z-58ee8190.json`, records a completed `READY` response, with 21 input and five output tokens. Its 4813.349-ms span includes credential retrieval. It establishes connectivity, not autonomous usefulness, customer readiness or benchmark model latency. The original 12-comment population remains separate and used the benchmark-authored MCP server.

8.9 First Real-Model Read-Only Run

The report `field-study-use-2026-10-19T06-33-57.736Z-0e238334.json` records six scheduled tasks using actual public GitHub sources and the existing Azure deployment. Field runtime source was `bc1d389`; 18 field-source entries accompanied the preserved 49-entry v0.2 inventory. The pre-request freeze, `field-study-freeze-2026-10-19T06-28-30.451Z-e3d388eb.json`, records task SHA-256 `990dbd74a66611cd2103cc537807ef7978343962fdcaae3d056559b783d4c0e9` and criteria SHA-256 `a845424d29788315b49c7540d4b134a87df75838608f0b3c0910267ff51df130`. Evidence was retained at `eff7bf6`. Tasks started in fresh contexts with fixed lanes and source bounds, without the fact rubric or another task's output. Source documents were Bench `cecde389` and Spec `a9b86f4`, distinct from execution/evidence commits.

All six loops returned a schema-valid JSON report, with every declared citation URL matching a successfully observed source URL. This check does not determine whether a cited line supports each assertion. The run used 35 model requests, 29 read calls, 155343 input tokens and 12540 output tokens, totaling 167883. Task spans sum to 326744.573 ms; the enclosing run takes 327262.889 ms. Recorded model-request, tool-handling and other task spans sum to 285544.690, 40481.360 and 718.523 ms. Earlier setup costs remain separate; request tokens include repeated context rather than distinct source bytes.

The outputs are concrete maintenance artifacts. T1 independently fetched 900348 report bytes and compared the digest to its adjacent checksum; optional release inspection retrieved metadata, not PDF bytes. T2 produced an ordered installation/readiness runbook, T3 a migration/rollback note, T4 an old-versus- official-server comparison, T5 condition-to-action recovery guidance and T6 a conformance claim/evidence crosswalk. These name the work products, not six completed quality or actual-use judgments.

Task	Facts	Met	Partial	Incorrect	Final bytes	Derived body grade
T1	4	4	0	0	7119	Complete
T2	5	5	0	0	8319	Partial
T3	5	5	0	0	8823	Partial
T4	4	3	0	1	8345	Incorrect
T5	6	5	1	0	7866	Partial
T6	6	5	1	0	8071	Partial
Total	30	27	2	1	All > 6000	1 complete, 4 partial, 1 incorrect

Table 12: First-run same-project AI-assisted fact review. The derived body grade does not imply original delivery compliance, which is 0/6. Human utility, review effort and actual maintainer use are unmeasured; facts are not independent tasks.

Retained response-contract deviation. Frozen tasks/rubric requested plain Markdown final text within 6000 UTF-8 bytes. The harness appended a JSON-envelope instruction targeting less than 7000 bytes, while the initial scorer checked fit against the profile's 8192-byte comment limit. All six bodies fit the latter but only T1's extracted Markdown fits 6000 bytes. All six complete final texts are JSON rather than plain Markdown and exceed 6000 bytes: original delivery compliance is 0/6, not 1/6. Five extracted bodies need length rework in addition to the shared format conflict. Conflicting instructions and the generous scorer are harness defects, not five independently attributable model failures. Schema and citation checks cannot supply missing quality credit. Inputs, outputs, criteria and source remain retained. The assessment in `analysis/review-first-field-use.json` is same-project AI-assisted rubric review, not independent human/external evaluation. Deterministic quotation, frozen-fact and byte checks make it auditable without making semantic labels independent.

The derived grade is incorrect if any required fact is incorrect; otherwise partial if any fact is unmet or the extracted Markdown exceeds 6000 bytes, and complete otherwise. Thus T1 is substantively/body-byte complete, not compliant with the original whole-response delivery contract. This derived assessment does not replace frozen criteria or the observed format failure.

T4 states that "no server, authentication, tests or provider operation was exercised in this task," a material scope error because its source reads used the official MCP server. Model-visible results omitted startup server metadata, confounding this finding with incomplete harness channel information rather than isolating model reliability. T5 incompletely identified the critical body-hash extension; T6 omitted baseline shared projection/controls and the absence of description exchange. Five reports require length rework, and T4 also contains the material error. These are report/fact judgments, not six observed human-utility successes.

The original replay passed its implemented consistency checks, but review found no independent reconstruction of complete event/source coverage. A strengthened post-review replay of the unchanged first-run report then returned `valid: true`: 242 journal records, 28 native-bound file observations including 27 retained texts, and one release-metadata observation accounted for 29 reads. File observations cover 20 distinct file URLs. Replay made zero model or provider requests; task execution made zero provider-write attempts.

The explicit `coverage_gaps` entry `release_native_envelope_not_retained` remains: the normalized release observation lacks its native raw envelope. No release asset was downloaded in this run. The version-2 reader retains this limitation, so native file binding and passing reconstruction are not full native release or asset verification.

Task	Lane	Body bytes	Loop	Wall, ms	In tok.	Out tok.	Grade
T1	SDK	5168	Completed	82818.211	25586	1973	Complete
T2	MCP	6272	Failed	51893.392	27035	2007	Partial
T3	SDK	7051	Failed	43425.587	17949	1909	Partial
T4	MCP	5997	Completed	52504.850	23387	1868	Complete
T5	SDK	6267	Failed	49715.020	34286	1888	Partial
T6	MCP	6454	Failed	51437.203	33700	1780	Partial

Table 13: Second-run observations on the same goals. All four failures are `report_bytes_exceeded`; every full JSON candidate is retained. Grades are separate same-project content review, not human acceptance.

8.10 Post-Review Read-Only Repeat

The report `field-study-use-2026-10-19T07-15-38.744Z-a76953f1.json` records a repeat at execution source `dff6ba7`, with evidence retained at `7306d8b`. The pre-request freeze, `field-study-freeze-2026-10-19T07-10-06.472Z-419977b3.json`, binds amendment 2, the same six goals, fixed facts, source documents and lanes, and fresh model contexts. Task and criteria hashes are respectively `2c7fdb9d451c5c6474f7a0688d92e22d3bc3b8b32d444ec03f96d768a777a0d5` and `ff7a7e6472414a916e39e377f3ed5e3181d9fa554d69ff0de848bc39c7a641ac`. Twenty field-source entries accompany the unchanged 49-entry v0.2 inventory.

The amendment explicitly requires a JSON envelope and limits its `report_markdown` field to 6000 UTF-8 bytes, recommending a 5000-byte target. It supplies bounded startup/channel metadata, not previous answers or rubric labels. This budget's scope differs from the original whole-final-text Markdown contract. Oversized candidates remain failed evidence rather than being truncated, automatically rewritten or resubmitted to the model.

The scorer counts schema/citation checks only for completed loops. Its two qualifying reports do not mean that the other four lacked JSON text or source observations. Per-task model-request/read counts equal those in the first run: 35 requests and 29 reads in total. Usage is 161943 input plus 11425 output tokens, totaling 173368. Task spans sum to 331794.263 ms; the enclosing run takes 332244.655 ms. No provider write was attempted. These are operational observations, not semantic-quality or human-utility grades.

The separate assessment in `analysis/review-second-field-use.json` reviews all six retained candidates against the same 30 facts: 28 met, two partial, none incorrect. Derived content/body-budget grades are two complete and four partial; failed loops remain failed. T4 now states, "All five source reads succeeded on the MCP lane," separating observed official-server reads from historical tests, mutations and remote authentication. Its earlier categorical scope error is absent under changed channel-information conditions. T5 still omits the required critical-extension mechanism for the body hash. T6 now names shared OPA, ledger and dispatcher, but still omits shared Python projection and the baseline's absence of an effect-description envelope.

All 29 citation entries were observed, including those in failed candidates. Their raw false/null/no-result scorer fields remain unchanged. The audit `field-study-review-audit-2026-10-19T07-23-39.012Z-f5138821.json` validates 30 literal report quotations, frozen-fact references, byte counts and grade arithmetic, not independent correctness of the same-project AI-assisted interpretation. Human acceptance, actual maintainer use, review effort and business utility remain unknown for every task.

The retained strict replay, `field-study-replay-2026-10-19T07-18-25.099Z-3c9423af.json`,

returns `valid: true` over 246 journal records, 28 native-bound file observations, 27 retained texts and one release-metadata observation: 29 reads, with zero model/provider replay requests. The release-native-envelope gap remains; no native release envelope or asset bytes were observed. Replay reconstructs the two completed/four failed split rather than upgrading failures to successes.

Review of the measured runner found a separate journal-failure gap: after `runAgent` returns `journal_error`, `run.mjs` can append `scheduled_task_end` and later seal a report as `recorded`. No such fault was observed in this run, but passing replay does not qualify that path. Post-measurement repair `c72369a` adds a runner-wide journal-failure latch and propagates agent journal errors before further task-end recording or sealing. Thirty-seven targeted regression tests passed after repair; 35 had failed against the old runner. This qualifies injected software failures, not cloud reruns, power loss or the measured `dff6ba7` revision retroactively.

The v0.3 local verification passed 1093 methods: 408 prior JavaScript, 115 Spec Python, 50 runtime Python, 510 field and 10 analysis. These are software-test methods, not workloads or extra human-utility evidence. Both normal strict replays remained valid at 242 and 246 records. No third model run is included.

These are six repeated goals, not twelve independent intents. Output contract, enforcement, channel information and replay changed together, while generation remained stochastic. Neither completion-count nor token/time differences isolate a causal amendment effect. The original six mechanical passes and amended two completions also apply different acceptance rules; they are not directly comparable utility rates.

8.11 Exploratory Publication Fit of Captured Reports

The report `field-study-publication-fit-2026-10-19T06-40-15.561Z-bdcae674.json` examines the first run's six outputs. The second fit report, `field-study-publication-fit-2026-10-19T07-18-12.145Z-b1330df7.json`, applies the same analysis to the repeat, including its four failed candidates. `analysis/publication-fit.mjs` constructs a standalone report/conclusion/all-citations body and a linked handoff containing the conclusion, immutable evidence URL and report-object digest. Each form is prepared by both strategies in the task's assigned lane: 24 related rows per run, not new independently sampled goals. Issue `2147483647` and principal `packaging-fixture` are synthetic placeholders. No OPA, ledger reservation, model call, provider request or public comment occurs.

The 6000-byte task budget and 8192-byte final native-body limit have different owners. Three first-run fully packaged bodies exceed the latter although all unpackaged bodies fit it. Both preparations reject those three and accept the other three; all six first-run linked handoffs of 613–787 bytes are accepted partial descriptions.

In the repeat, all standalone bodies (6486–7834 bytes) and linked handoffs (743–928 bytes) fit: all 24 preparations are `accepted_partial` with authority not evaluated. Four underlying task bodies nevertheless exceed 6000 bytes and remain failed. There is no contradiction: task-budget compliance, native-body fit, semantic quality and permission are different predicates. Neither fit analysis invokes OPA, a ledger, provider or model.

The link preserves the full artifact instead of truncating it. It selects evidence commit `eff7bf6` for run one or `7306d8b` for run two, each locally verified to contain the corresponding report bytes. The embedded task digest hashes UTF-8 bytes of `JSON.stringify` applied to the parsed per-task report, distinct from the whole-run checksum and comment-body SHA-256. This packaging identity is not a new core canonicalization rule. Acceptance covers the handoff body, not the linked report's truth

Run	Task	Full bytes	Full result, both	Link bytes	Link result, both
1	T1	7086	accepted partial	774	accepted partial
1	T2	8264	invalid native input	737	accepted partial
1	T3	8798	invalid native input	777	accepted partial
1	T4	8356	invalid native input	787	accepted partial
1	T5	7835	accepted partial	708	accepted partial
1	T6	8050	accepted partial	613	accepted partial
2	T1	6486	accepted partial	878	accepted partial
2	T2	7318	accepted partial	789	accepted partial
2	T3	7834	accepted partial	743	accepted partial
2	T4	7210	accepted partial	928	accepted partial
2	T5	7370	accepted partial	812	accepted partial
2	T6	7528	accepted partial	796	accepted partial

Table 14: Preparation-only publication fit. Invalid results are `invalid_input/native_input`; accepted results are `accepted_partial/declared_partial`. All retain authority not evaluated.

or availability. The finding is a concrete composition constraint and equal preparation behavior, not improved quality, policy approval or a competing-system win.

8.12 Owned Delivery, Real Process Failure, and Recovery

The owned-write report `owned-write-study-2026-10-19T13-50-36.691Z-ff5e0a2c.json` continues the amended cohort without new model answers. It freezes execution source `f93b1cfc bf2c2665b4acc7c5dc8fb92d1ba8d6b6`, the same Spec revision, 78 source entries, and the exact committed task and review records. Both source checkouts were clean. T1 and T4 qualify; four task-budget failures remain withheld before authorization. This is not a third model trial or a repair of the failed outputs.

Apparatus and observations. The sole destination is the newly created owned issue #2; older issue #1 is untouched. The plan permits at most twelve application-level comment calls across processes but expects five objects. The two reports retain their original conclusions and source citations, with explicit same-project review/evidence references added. Final bodies contain 6917 and 7659 bytes, within this profile's 8192-byte bound, not a claim about GitHub's platform maximum. The six fault conditions use distinctly labelled synthetic bodies, not additional useful model tasks.

Each initial execution, reconciliation and duplicate probe starts a fresh OS process and journal; twenty distinct worker PIDs are retained. The supervisor kills only its owned process groups. Four initial scenarios select SDK and four select official GitHub MCP v2.0.2 over stdio. Identity, preflight and observation use the SDK in both lanes. Recovery/probe processes expose no native mutation transport. Both protocol and complete structured preparation check each native proposal against a literal seven-field reference, but only the selected path dispatches. All eight initial proposals match in both preparations. Repeated shadow checks during recovery/probes are not independent cases or live trials.

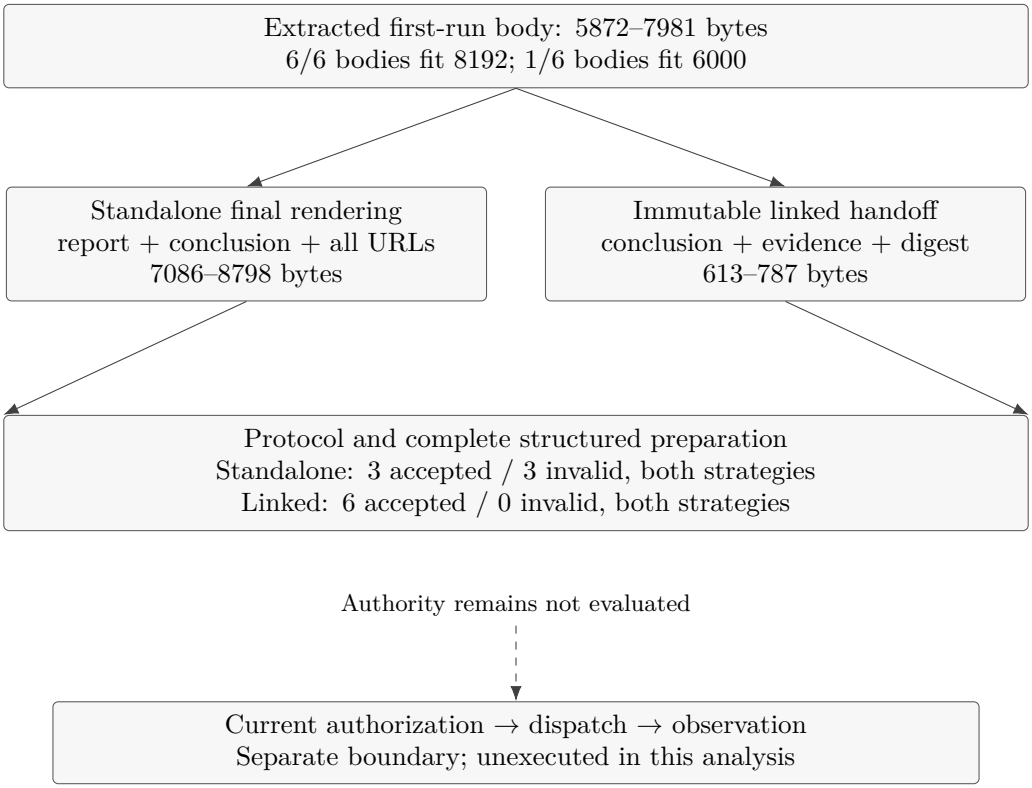


Figure 8: First-run rendering before mapping; authorize separately. Final native-body bytes, not an earlier model field or token ceiling, determine profile fit. The dashed edge is not an executed policy/provider path.

Table 15: Frozen owned-delivery conditions and observations. P = protocol; S = complete structured preparation. All eight duplicate probes make no native comment call.

Scenario	Lane/path	Initial outcome	Fresh recovery	Objects
Release-integrity report	SDK/P	Confirmed	Not needed	1
MCP compatibility report	MCP/S	Confirmed	Not needed	1
Revoke after preflight	MCP/P	Withheld: revoked	Not needed	0
Narrow body limit after preflight	SDK/S	Withheld: body too large	Not needed	0
Kill after inflight persistence, before call	SDK/P	SIGKILL	Unknown: no unique readback	0
Kill after actual MCP reply, before receipt	MCP/P	SIGKILL	Confirmed current content	1
Discard SDK reply, expire timer	SDK/S	Unknown	Confirmed current content	1
Discard MCP reply, close stdio	MCP/P	Unknown	Confirmed current content	1

The two authority changes occur after provider preflight and before the final policy load. Current deny prevents both mutations, rather than reporting a later audit anomaly. In the first crash,

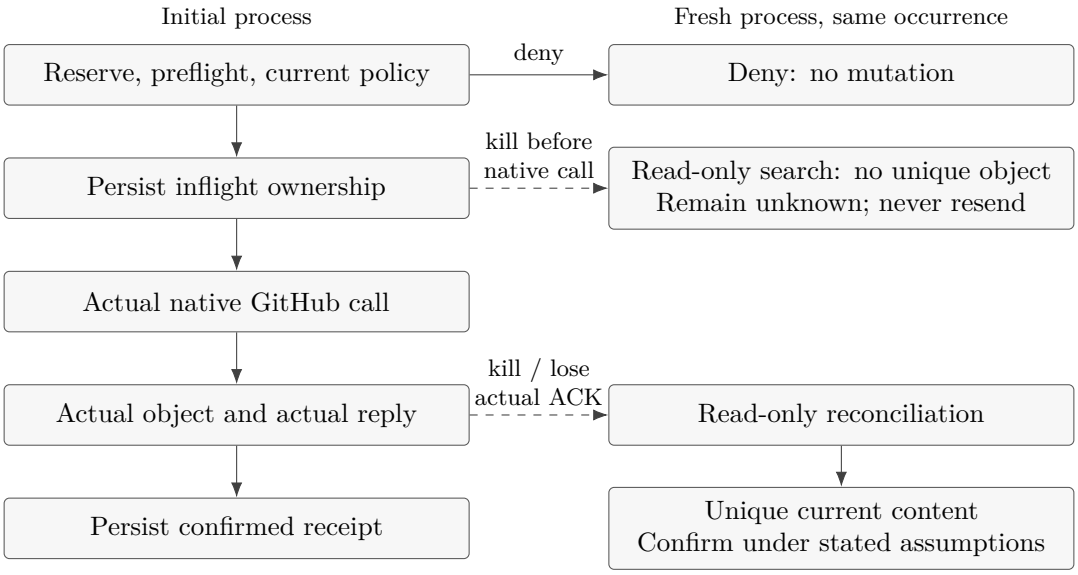


Figure 9: Executed fault cuts and the information available after restart. No fabricated provider success response replaces a real call. Current-content recovery depends on the single-writer observation assumption; it is not atomic remote revocation or proof of all provider effects.

durable inflight state exists but no native writer call occurred. The restarted gate finds no unique current match, retains occurrence ownership and returns unknown. This is intentional: missing evidence must not become permission to send, even though the experiment supervisor knows where it stopped the earlier process.

In the post-reply crash, an actual official-server reply is journaled before SIGKILL but no confirmed gate receipt exists. The timeout control observes a successful SDK reply, withholds it from the gate and waits for a real 500 ms timer. The disconnect control captures an actual mutation JSON-RPC response, discards it and closes real stdio. These are deliberate acknowledgment-delivery faults, not spontaneous provider or internet outages. Three subsequent processes use only GET-based reconciliation to confirm current matching objects; none resends a POST to obtain success.

Cost without a fabricated speed contest. The complete supervisor run took 52436.648 ms, including fresh workers, paired shadow preparation, current-source checks, networking, fault synchronization and final observations. Earlier issue creation and setup are excluded, separate costs. Ordinary gates took 3589.962 ms (SDK/P) and 4775.226 ms (MCP/S); worker lifetimes were 4049.513 and 5346.507 ms. Current OPA cost was 20.581 and 20.339 ms; dispatch spans were 1551.673 and 1827.885 ms, with readback 876.455 and 599.670 ms. Consumer cost is nested within mapping, not added twice. Source checks and journal work immediately before sending are included in dispatch spans. We do not report only a fast local predicate.

One live path per different scenario cannot identify a causal protocol/baseline cost difference. Shadow order is always protocol then structured, so order and warm-up confound those retained raw samples. The earlier offline study's 3.97% overhead remains the bounded comparison, not a live speedup claim. No p99, enterprise SLA, natural outage rate or statistical zero-risk bound is inferred.

Phase class	n	Gate/recovery, ms	Whole worker, ms
Ordinary SDK/P	1	3589.962	4049.513
Ordinary MCP/S	1	4775.226	5346.507
Pre-send unknown recovery	1	1089.062	1507.631
Confirming read-only recovery	3	898.529–2008.267	1323.215–2437.639
Duplicate probes, no send	8	34.610–41.902	446.024–480.242

Table 16: Observed single-run cost boundaries. Ranges are minima/maxima, not percentiles, population intervals, throughput or an SLA.

Retained failure and correction. The first attempt created issue #2 but stopped before reserving any occurrence or comment-call permit. Its raw record remains public in `reports/owned-write-pre-reservation-failure.md`. The Python consumer's parsed projection had a null JavaScript prototype; the literal reference used a normal prototype. Host-object equality rejected identical JSON values before dispatch. A regression reproduced the failure; the corrected comparison uses the profile's canonical JSON digest and still rejects a changed field. The new manifest binds that failed attempt and adopts the same unchanged empty issue only after fresh inspection. It neither retries an uncertain mutation nor removes the failure. Further review hardened owned-child cleanup, actual MCP-close error propagation and pre-call source/OPA identity checks. Those local fixture regressions are software qualification, not additional GitHub incidents.

Audit boundary. The replay `owned-write-replay-2026-10-19T13-50-51.561Z-2e5d127c.json` reconstructs twenty worker journals plus the supervisor: 283 records, five call permits, five current objects, eight no-send probes, three confirmed recoveries and one retained unknown. It checks frozen inputs, historical source blobs, literal projections, current policy, signals, ledger continuity, native replies and final content. Missing/corrupt journals and changed outcome/permit records are refused. Readers can independently fetch public objects, but this does not authenticate every operator-supplied historical event. The separate audit `owned-write-audit-2026-10-19T13-58-07.840Z-d8467612.json` made six credential-free GETs at 13:58:03–13:58:07 UTC on 19 October. The issue and all five exact comment IDs returned HTTP 200 and matched committed body hashes, identifiers, URLs and authors. Raw replies, hashes and observation times are retained: new current-state observation, not a new write or proof of the earlier execution time.

The study connects read-only report construction with an observed native effect for this profile. It does not establish broad model reliability: two of six candidates qualified and independent human usefulness is unknown. A dedicated issue, same-host durable ledger, single writer, trusted adapter and bounded partial coverage remain material conditions. Last-check-to-remote-commit policy changes, another same-account writer, lost storage and notification descendants still need separate treatment.

9 Adoption, Rights, and Publication Boundary

Adoption is staged and interface-only: bounded offline implementation, non-dispatching comparison, then a separately authorized owned-resource test. No OSuite endpoint, account, module, purchase, or code is required. The adoption quickstart documents implemented installation, offline, replay, and authorized-live interfaces with automatic read-only lost-ACK recovery. Supplied unit tests and

source inspection do not establish completed measurements, customer deployment readiness, or a reason to remove existing controls.

The public RFC asks for concrete tuple-loss, drift, unknown/failure, freshness, recovery, and fair-baseline tests. Protected inputs and outcomes must not be posted publicly. Unrelated protected collaboration data is excluded. Outreach copy is unsent material, not evidence of participation or endorsement.

The repositories retain MIT for selected original contributions; this document does not change that license. Third-party sources and outside submissions retain their applicable rights and need appropriate authority for incorporation. MIT publication is not patent/legal clearance, a separate patent covenant, certification, standards-body approval, or another party's authorship consent. This institutional working paper is for public review, not an automatic formal submission. Formal submission or an individual-author byline requires separate confirmation. No external coauthor, endorsement, or custodial appointment is implied; outreach copy remains unsent.

10 Conclusion

Action-Effect makes native capture, coupled meaning, evidence, current authority and observation explicit and executable. The v0.2 results establish exact mapping/decision parity, controlled read-only recovery and 3.97% higher accepted-input median cost. Real-model maintenance shows that completed loops and source citations can coexist with a broken output contract and material report errors. The amended repeat retained four task-budget failures even though all its native preparations fit; profile acceptance is not task completion. Final rendering changes profile fit: three first-run fully cited bodies exceeded the limit while linked handoffs fit in both strategies. The owned continuation delivered the two eligible reports using real SDK and official-MCP writes, denied current revoked/narrowed authority, and recovered three delivered effects without resending while preserving one unknown. Eight duplicate probes made no further calls. The artifact is a reconstructable interchange, measurement method and conditional recovery contract. Broader portability and useful deployment remain empirical questions under the same complete-baseline and retained-failure standards.

A Complete Profile Construction Example

This is an illustrative local construction, not another execution or permission to write to issue 1. The principal is not an authenticated account. Let P be the released comment manifest and use these complete input objects:

```
{
  "native": {
    "lane": "sdk",
    "call": {"owner": "OndCo", "repo": "Action-Effect-Bench",
             "issue_number": 1, "body": "abc"}
  },
  "context": {
    "principal": "example-maintainer",
    "data_class": "synthetic-public",
    "implementation": "github-comment-bridge/1"
  }
}
```

The projection's operation is `create_issue_comment`, repository `OndCo/Action-Effect-Bench`, issue 1, class `synthetic-public`, and byte count 3. Its endpoint is https://api.github.com/repos/OndCo/Action-Effect-Bench/issues/1/comments. The exact content hash is `ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad`.

Define $B(name, x)$ as a binding whose reference is `urn:action-effect:github-comment:` followed by *name*, with algorithm `sha-256` and value $\mathsf{H}(x)$. Define $K(ns, v)$ as a known slot with namespace ns and value v. Table 17 specifies all fields without repeatedly expanding immutable namespace URLs. Manifest fields mean literal configured values, never network lookups.

Table 17: Complete description construction. Symbolic names are strings unless explicitly objects or arrays. This is a recipe, not abbreviated wire JSON.

Field	Construction
Wire/profile	P.core_spec and P.id
Profile digest	Algorithm sha-256, value $\mathsf{H}(P)$
Record ID	Local record URN prefix plus $\mathsf{H}(\{\text{native} : N, \text{context} : C\})$
Invocation/context	$B(\text{invocation}, N)$; $B(\text{context}, C)$
Implementation	$B(\text{implementation}, \{\text{id} : P.\text{implementation}, \text{kind} : \text{declaration}\})$
Mapping	ID P.id plus fragment mapping, revision string 1, digest $\mathsf{H}(P)$
Coverage	Direct, partial; one assumption: Declared direct GitHub comment only; provider descendants not bounded. Refs: declaration.
Effects	One comment ID; category and operation from P; modality may; evidence refs declaration
Resource	$K(P.\text{resource_namespace}, \text{OndCo/Action-Effect-Bench\#1})$
Destination	Known slot with manifest destination namespace and projection endpoint
Data class	Known slot with manifest data namespace and synthetic-public
Quantity	One body-byte quantity, value string 3, unit byte, scale string 0
Evidence	One declaration: source P.id, local declaration artifact reference, digest of named native/context/profile-digest object, scope direct-request-declaration
Unknowns	One unobserved-descendants item, exact manifest unknown scope
Relations	Empty array

Field	Construction
Critical extension	One manifest body-digest extension ID
Extension payload	Exactly that key, holding the body SHA-256

Arrays retain the stated order. There are no added observations or fields. The example's digest values below are arithmetic identities, not experimental results. The equivalent MCP capture has another invocation hash while retaining the same projected direct operation.

Table 18: Illustrative canonical preimage digests.

Preimage	SHA-256
Profile P	`c4c062d8181d3e0d2cd04a0667ec6711d159bd61db5eeeae95217918e80350b3`
Context C	`818d64bb2144b707c0549baabfcbde04bad3f5d3ced3acc2ac69a018c0bf8dc2`
SDK capture	`e3810e3f794cb56e1ff7e7bbb7f37331ef693d88001f685ba5d86b7c4b09d36b`
MCP capture	`aac012db8f99856868110153e6b5b86ee68f130a987ca44e1622c3345b276d71`
Implementation	`73f14cd57f597e916d5ff8c9f513029a488a528a73b7e520d641edeb778d607f`
SDK pair	`eed927eb65697081acdcc0c9b1bfca8e0bdab0696f312142298ac306221ab839`
SDK declaration	`533a57eddceabfb41a049017af722f16aa2ff66a1e8b27a9d5cb7b346692df2f`

B Core Obligation Index

The pinned specification is normative, not this summary. The index maps each CORE ID to the prototype's boundary. A row is not full conformance evidence or a replacement for the applicable design vectors.

Table 19: All 28 core obligations and their implementation boundaries.

CORE	Obligation	Prototype boundary
001	Separate stages	Processing, authorization and outcome fields
002	Material trust basis	Configured declaration and provider identity, no general attestation
003	Coupled tuples	Exact projection and one-row grant matching
004	Unique/resolved references	Local checks; no first/last-match guessing
005	Versioned category semantics	Finite installed profile, not automatic taxonomy resolution

CORE	Obligation	Prototype boundary
006	Modality/scope/coverage	One may effect, direct and partial
007	Bound establishment	No complete effect bound established
008	Known effects plus unknowns	Provider-descendant unknown retained
009	Strict encoding and JSON	Byte parsers and ordered native validation
010	Exact quantities	Byte strings, unit/scale; no currency conversion
011	Profile-bound identity	Exact case-sensitive identity and manifest digest
012	Explicit digest preimages	Complete JSON objects versus body bytes
013	Mapping applicability	Finite implementation declaration
014	Invocation/context association	Complete description comparison; separate occurrence
015	Source/artifact/scope of evidence	Declaration, not external analysis or review
016	Condition/error semantics	No general condition-language execution
017	Separate failure and decision	Unsupported/error is not semantic denial
018	Withhold unmet obligations	Ordered policy and gate; preserve known denial
019	Immutable revision/withdrawal	Versioned artifacts, not operating registry
020	Acceptance reuse dependencies	No acceptance cache or selective-invalidation claim
021	Signed distribution freshness	Not implemented by repository hashes
022	Current authority on reuse	Reload before new send; recovery grants nothing
023	Critical extensions	Required body hash; unknown critical semantics refused
024	Finite limits	Parser, schema, worker, OPA and listing bounds
025	Untrusted references	No profile-triggered arbitrary fetch/execution
026	Telemetry/minimization	Selected synthetic reports, not automatic anonymization
027	Bound observations	Current-content receipt, conservative recovery
028	Scoped conformance claims	Same-project bounded report, no certification

C Processing Limits and Refusal Ownership

These are resource/interpretation bounds, not measured capacity claims. Exceeding a bound must not produce truncated success. Values describe the pinned runtime and core utilities rather than every possible profile.

Table 20: Implemented finite limits and the responsible refusal stage.

Boundary	Limit/check	Refusal
Core frame	1 MiB, nesting 64	Parser limit failure
Semantic arrays	256 effects/evidence/unknowns, 1024 relations	Invalid structure
Quantities	78 coefficient digits, schema scale 0–18	Structural/profile rejection
Body	1–8192 UTF-8 bytes	Invalid native input
Issue	Integer 1–2147483647	Invalid native input
Python child	10 s, 2 MiB output buffer	Consumer process failure
OPA child	2 s, 65536 output bytes, 256 KiB input	Explicit engine/input failure
Grants	At most 1024 exact objects	Invalid policy
Provider HTTP	15 s, redirects refused	Preflight failure or delivery unknown
Listing	Five 100-entry pages, final short page	Incomplete listing failure
Occurrence	Path-safe, 1–120 characters	Ledger/configuration failure
Replay files	32 MiB bounds, constrained journal path	Integrity failure

Core syntax can represent an unknown quantity when a supported profile defines it. The comment native contract requires concrete body and issue values, so missing ones are invalid, not invented unknowns. Syntax capability does not force every domain to treat a missing field as meaningful uncertainty.

D Artifact and Replay Procedure

Measured v0.2 source remains Bench `9f638df` and Spec `a9b86f4`. Later documentation or field-study commits do not relabel those observations. Use lowercase sibling public clones and the pinned Spec checkout. Never change the executable inventory merely to make a retained report pass.

Table 21: Retained artifacts under reports. JSON reports have adjacent checksums; the field journal is hash-bound by its use report.

Filename	Scope
`runtime-pilot-offline-2026-10-19T04-23-54.715Z-6bb423.json`	216 rows, 26 pure and 26 diagnostic controls
`runtime-pilot-replay-2026-10-19T04-24-39.129Z-61da80.json`	216 row and 52 reference/diagnostic checks
`runtime-pilot-live-2026-10-19T04-26-11.245Z-731e54.json`	32 slots, 12 receipts
`runtime-pilot-live-attempt-338f0652-715c-4095-84df-4f19d8e2f8c7.jsonl`	66 records: begin, schedules, results, end

Filename	Scope
runtime-pilot-replay-2026-10-19T04-26-57.109Z-690395.json	32 rows and 12 anonymous GET matches
runtime-pilot-offline-2026-10-19T03-40-48.927Z-769420.json	First failure; exact old executable tree not archived
field-study-freeze-2026-10-19T06-28-30.451Z-e3d388eb.json	First-run task/criteria and source freeze
field-study-use-2026-10-19T06-33-57.736Z-0e238334.json	Six tasks, traces, mechanical checks and usage
field-use-2026-10-19T06-28-30.337Z-3ede5a.jsonl	242 records, report-bound hash
field-study-publication-fit-2026-10-19T06-40-15.561Z-bdcae674.json	24 preparation-only rows
field-study-freeze-2026-10-19T07-10-06.472Z-419977b3.json	Amendment-2 source/task/criteria freeze
field-study-use-2026-10-19T07-15-38.744Z-a76953f1.json	Same goals: two completed, four retained byte-budget failures
field-use-2026-10-19T07-10-06.329Z-1a193a.jsonl	246 records, second-report-bound hash
field-study-replay-2026-10-19T07-18-25.099Z-3c9423af.json	Strict repeat replay, release-envelope gap retained
field-study-publication-fit-2026-10-19T07-18-12.145Z-b1330df7.json	24 preparation rows from all six repeat candidates
field-study-review-audit-2026-10-19T07-23-39.012Z-f5138821.json	30 literal quotes, frozen facts, bytes and grade arithmetic
owned-write-study-2026-10-19T13-32-22.199Z-ab5303e5.json	Retained pre-reservation failure; no comment calls
owned-write-study-2026-10-19T13-50-36.691Z-ff5e0a2c.json	Two ordinary deliveries, six faults, twenty workers
owned-write-20261019-b.*.jsonl	Twenty worker journals and supervisor; 283 records
owned-write-replay-2026-10-19T13-50-51.561Z-2e5d127c.json	Source, plan, policy, process, receipt and no-resend checks
owned-write-audit-2026-10-19T13-58-07.840Z-d8467612.json	Timing arithmetic and six anonymous current-content GETs

The v0.2 digest inventory and source-blob arithmetic appear in `measurement-contract.md`, `measurement-audit.json` and `sources.json`. File hashes identify raw bytes; description hashes identify canonical JSON values. Reformatting a report changes its file hash even if all parsed values remain equal.

The first field-use report has SHA-256 `cdc07b7a10000a4fe96152a6a460f5633c3403bcc7c0cd8f3f68f8e1a5cbe9fc`; publication-fit has SHA-256 `7bf1f16c13bd92b5db2952ea045487`

0163e3522cf8e5d8524e68c7579ddb3bc6. Second use, fit and strict-replay SHA-256 values are, respectively, e6d9fe0e27ca93932fb9102cc5d5b7f16e4c7c138cd72c79eeeaaf1cb2961999, 3b501f6261821e5bd0c3d7901c090587c519e2c195cdcfa86e67282633a260e5, and b9d854f40a79c9e68a3447470b2b7af768c198d0d7b4ab17d0f82a641f1048b9. The second review in analysis/review-second-field-use.json has SHA-256 0f5b7954f549c1e184f5ece621126234f2c1c8034175391b1f1354ef2eca2682; its audit report has SHA-256 a6c3490fcdd8957ec14a980ed709c18499bcdac6eb679ce1adadacd960186194.

At a checkout matching the report's executable inventory:

```
node runtime-pilot/replay.mjs reports/runtime-pilot-offline-2026-10-19T04-23-54.715Z-6
    bb423.json
node runtime-pilot/replay.mjs reports/runtime-pilot-live-2026-10-19T04-26-11.245Z-731
    e54.json --github
```

Replay checks checksums and exact source inventory first. Offline integrity checks scheduled keys, literal inputs/context/policy/references, derived counts and timing summaries before rerunning mapping and OPA at the matched source. Pure and deliberately lossy controls remain separate. Live replay additionally checks plan, exact reasons/outcomes, discard witnesses and journal consistency. Duplicate rows are not reauthorized or converted to new grants.

With -github, receipt IDs are deduplicated and anonymous GETs compare current content. No POST or model request occurs. Deletion, edits, rate limits or missing artifacts remain failed/unavailable checks. The newly written replay timestamp is not the original event time. Reports and checksums can be forged together; retained trusted pins and custody are needed beyond consistency.

The first field report retains its bc1d389 execution runtime and frozen task/criteria files. Its command is node field-study/replay.mjs REPORT.json; the checker loads recorded execution sources rather than relabeling the run with its own later revision. Section 8.9 distinguishes the original consistency pass, stronger post-review reconstruction and retained native-release gap. Checker revisions do not authorize historical evidence edits.

Owned-write source is f93b1cfcbf2c2665b4acc7c5dc8fb92d1ba8d6b6. Its report and first successful replay have SHA-256 735bfdfc5c5ae993f64b6374ec599bf4ee76429dbcf995305cc2c9c3832ff6d7 and 50a150f868ae00b065bbae35b1f782988a474cb0533cc526746f7f217302d384. The earlier pre-reservation failure is also required by the new manifest. Run node field-study/write-replay.mjs REPORT.json from a full clone with the recorded Git history. Replay reads old sources as data rather than checking them out or repeating mutations. The live commands in field-study/OWNED-WRITE.md are not replay commands.

E Compatibility and Deployment Decisions

Before adding a domain, define capture, exact quantity/identity semantics, permitted context acquisition, evidence reference rules, supported scope and required unknowns. Implement an independent consumer and a competent direct adapter. Arbitrary JSON keys or unknown-URL resolution do not make a profile generic.

Migration checks wire, profile, processing contract, installed implementation and policy separately. Unknown critical semantics require refusal, not removal and retry. An unavailable old revision cannot be interpreted under new rules without changing the claim. Historical reports name their executed versions.

Product use requires measured support for actual domains, credentials, durable ownership, operator recovery and task utility. The synthetic-public experiment's partial-direct exception cannot satisfy a customer's complete-descendant policy. The one-host ledger also needs replacement or qualification before multi-host use. These are engineering acceptance criteria, not permissions silently created by a public manuscript.

References

[S1] Model Context Protocol. *2025-11-25 schema*, selected source commit `c4c367f9f58296a7053f5c78a52fd02bfbb56a49`; tool annotations. https://github.com/modelcontextprotocol/modelcontextprotocol/blob/c4c367f9f58296a7053f5c78a52fd02bfbb56a49/schema/2025-11-25/schema.ts.

[S2] OpenID Foundation. *Authorization API 1.0*. Final, 11 January 2026, sections 5–7. https://openid.net/specs/authorization-api-1_0.html.

[S3] T. Lodderstedt, J. Richer, and B. Campbell. *OAuth 2.0 Rich Authorization Requests*. RFC 9396, May 2023, sections 2.2, 6.1, 7, and 9. https://www.rfc-editor.org/rfc/rfc9396.txt.

[S4] Open Policy Agent. *REST API documentation*, Data API. Documentation commit `7bcf86187363b525802096f4668c2db1e058a831`, not an engine-run pin. https://github.com/open-policy-agent/opa/blob/7bcf86187363b525802096f4668c2db1e058a831/docs/docs/rest-api.md.

[S5] Open Policy Agent. *Policy Language documentation*, objects, defaults, and built-in errors. Same documentation commit as S4. https://github.com/open-policy-agent/opa/blob/7bcf86187363b525802096f4668c2db1e058a831/docs/docs/policy-language.md.

[S6] Cedar. *Authorization documentation*. Documentation commit `3a37098eba6e02e716949cfc92b889b38db25602`; no Cedar execution claim. https://github.com/cedar-policy/cedar-docs/blob/3a37098eba6e02e716949cfc92b889b38db25602/docs/collections/_auth/authorization.md.

[S7] Cedar. *Policy validation against schema*. Same documentation commit as S6. https://github.com/cedar-policy/cedar-docs/blob/3a37098eba6e02e716949cfc92b889b38db25602/docs/collections/_policies/validation.md.

[S8] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett. *Best Current Practice for OAuth 2.0 Security*. RFC 9700, January 2025, section 2.3. https://www.rfc-editor.org/rfc/rfc9700.txt.

[S9] W3C. *PROV-DM: The PROV Data Model*. Recommendation, 30 April 2013. https://www.w3.org/TR/2013/REC-prov-dm-20130430/.

[S10] A. Rundgren, B. Jordan, and S. Erdtman. *JSON Canonicalization Scheme (JCS)*. RFC 8785, Informational, June 2020. https://www.rfc-editor.org/rfc/rfc8785.txt.

[S11] GitHub. *REST API endpoints for issue comments*. Selected documentation API version `2022-11-28`; create/get comment and notification notice. https://docs.github.com/en/rest/issues/comments?apiVersion=2022-11-28.

[S12] R. Fielding, M. Nottingham, and J. Reschke. *HTTP Semantics*. RFC 9110, June 2022, section 9.2.2. https://www.rfc-editor.org/rfc/rfc9110.txt.

[S13] GitHub. *Official REST API description source commit* `755ee33b4a81d51f6f07a902cce26cbec3b822f2`. Commit identity verified; not the extracted operation bytes or independent governance labels. https://api.github.com/repos/github/rest-api-description/commits/755ee33b4a81d51f6f07a902cce26cbec3b822f2.

[S14] GitHub. *Server Configuration Guide*, tool selection and read-only mode; GitHub MCP server v2.0.2, source `eb47a99ddb866ca2b8a162920e6bda9521f33ebb`. https://github.com/github/github-mcp-server/blob/eb47a99ddb866ca2b8a162920e6bda9521f33ebb/docs/server-configuration.md. Retrieved `2026-10-19T06:48:36.127Z`.

[S15] GitHub. *Typed tool schemas*, same pinned server source as S14. https://github.com/github/github-mcp-server/blob/eb47a99ddb866ca2b8a162920e6bda9521f33ebb/docs/typed-tool-schemas.md. Retrieved `2026-10-19T06:48:36.386Z`.

[S16] GitHub. *Issue tool implementation*, `AddIssueComment`; same pinned server source as S14. https://github.com/github/github-mcp-server/blob/eb47a99ddb866ca2b8a162920e6bda9521f33ebb/pkg/github/issues.go. Retrieved `2026-10-19T06:48:37.062Z`.

[S17] Microsoft. *Use the Azure OpenAI Responses API*, resource-specific `/openai/v1/responses` endpoint. https://learn.microsoft.com/en-us/azure/foundry/openai/how-to/responses. Retrieved `2026-10-19T06:50:56.789Z`. Deployment version/region are run observations rather than facts inferred from this general API guide.

The S1–S13 retrieval inventory remains in `sources.json`; S14–S17 pin the additional integration sources above. Raw third-party captures are not redistributed. This standalone file embeds its bibliography, diagrams and appendices and needs no companion files to compile.